

Báo cáo HW02 Domain Testing

Thông tin sinh viên

- Họ và tên: Lê Mai Hoài Bảo
- MSSV: 23127326

Báo cáo kiểm thử miền (Domain testing)

Pool A: FR-02: Đăng nhập & Khóa tài khoản

Bước 1: Xác định các biến Input và Output (I/O Variables)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các biến vào/ra của tính năng **FR-02: Đăng nhập & Khóa tài khoản**, em đã thực hiện các bước phân tích sau:

- Phân tích đặc tả nghiệp vụ (Specification Analysis):** Đọc kỹ tài liệu đặc tả yêu cầu hệ thống [README.md](#), chức năng này yêu cầu người dùng nhập **email** và **password** qua form giao diện. Đồng thời, nghiệp vụ có yêu cầu nâng cao: ghi nhận số lần đăng nhập sai liên tiếp và tạm khóa tài khoản 30 giây nếu đăng nhập sai liên tiếp từ 3 lần trở lên.
- Xác định biến đầu vào trực tiếp (Direct Inputs):** Hai trường người dùng tương tác trực tiếp là **email** (chuỗi ký tự, có validate HTML5 format trên UI) và **password** (chuỗi ký tự ẩn).
- Xác định biến đầu vào trạng thái (System State Inputs):** Do logic khóa tài khoản phụ thuộc vào lịch sử đăng nhập trước đó và thời gian khóa, hệ thống bắt buộc phải lưu trữ trạng thái: số lần đăng nhập sai liên tiếp trước đó (**consecutive_failed_logins**) và trạng thái khóa của người dùng (**lockout_state** / thời gian trôi qua kể từ lúc bị khóa). Đây chính là các tham số đầu vào ẩn quyết định luồng xử lý tiếp theo của ứng dụng.
- Xác định biến đầu ra (Outputs):** Ở mức API, hệ thống trả về mã trạng thái HTTP (**http_status_code**) và dữ liệu JSON (**api_response_payload** chứa token khi thành công hoặc thông báo lỗi bảo mật chung khi thất bại). Ở mức giao diện, hệ thống hiển thị thông điệp lỗi (**ui_message**) trên nút submit và thực hiện hành động điều hướng hoặc vô hiệu hóa form (**ui_action**).

1. Các biến đầu vào (Input Variables)

Bao gồm các biến do người dùng nhập trực tiếp từ giao diện/API (Direct Inputs) và các biến trạng thái hệ thống đóng vai trò làm tham số đầu vào cho logic xử lý (System State Inputs):

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	email	Direct Input	String	- Chuỗi ký tự. - Phải sử dụng type="email" (validate HTML5 format, e.g., user@domain.com).	Địa chỉ email dùng để đăng nhập.
2	password	Direct Input	String	- Chuỗi ký tự. - Không hiển thị rõ trên giao diện (type="password").	Mật khẩu dùng để đăng nhập.
3	consecutive_failed_logins	State Input	Integer	- Số nguyên không âm (>= 0). - Tăng thêm 1 sau mỗi lần đăng nhập thất bại. - Đặt lại về 0 khi đăng nhập thành công.	Số lần đăng nhập sai liên tiếp của tài khoản.
4	lockout_state	State Input	Enum	- Trạng thái: Đang bị tạm khóa (Locked) hoặc Đang hoạt động (Active). - Thời gian khóa: 30 giây (môi trường demo) nếu đăng nhập sai liên tiếp >= 3 lần.	Trạng thái tạm khóa của tài khoản và thời gian đếm ngược (nếu bị khóa).

2. Các biến đầu ra (Output Variables)

Bao gồm các phản hồi từ hệ thống (API Outputs) và thay đổi trạng thái/giao diện người dùng (UI Outputs):

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	http_status_code	API Output	Integer	- 200 OK (Đăng nhập thành công). - 400 Bad Request / 401 Unauthorized / 403 Forbidden (Đăng nhập thất bại / Tài khoản đang bị khóa).	Mã trạng thái HTTP trả về từ backend API.
2	api_response_payload	API Output	JSON Object	- Thành công: Trả về chuỗi JWT token và thông tin đối tượng user (id, name, email, role). - Thất bại: Trả về thông báo lỗi phù hợp (không tiết lộ chi tiết nguyên nhân đăng nhập sai để bảo mật).	Nội dung JSON phản hồi từ API.
3	ui_message	UI Output	String	- Thành công: Không hiển thị lỗi. - Thất bại: Hiển thị thông báo lỗi phù hợp trên nút submit.	Thông điệp thông báo hiển thị trên giao diện người dùng.

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
4	ui_action	UI Output	Enum	- Redirect : Chuyển hướng người dùng sang trang tương ứng và lưu trữ JWT Token phía client. - Lock : Vô hiệu hóa nút Đăng nhập và hiển thị thời gian chờ (30 giây) nếu tài khoản bị tạm khóa.	Hành động điều hướng và cập nhật trạng thái giao diện phía client.

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để thực hiện kỹ thuật Phân hoạch tương đương cho tính năng **FR-02: Đăng nhập & Khóa tài khoản**, em đã áp dụng các bước có hệ thống sau:

1. **Phân tích điều kiện đầu vào/đầu ra:** Dựa trên danh sách các biến đã xác định ở Bước 1, em tiến hành phân tích các điều kiện ràng buộc đối với từng biến dựa trên tài liệu đặc tả hệ thống.
2. **Xác định các lớp tương đương Valid (EC hợp lệ) và Invalid (EC không hợp lệ) từ bên ngoài:**
 - Đối với *Email*: Phân hoạch dựa trên sự tồn tại của email trong CSDL (hợp lệ/không tồn tại) và tính hợp lệ về mặt định dạng chuỗi (định dạng email chuẩn so với sai định dạng hoặc để trống).
 - Đối với *Mật khẩu*: Phân hoạch dựa trên độ trùng khớp thông tin mật khẩu của tài khoản (mật khẩu đúng so với mật khẩu sai hoặc để trống).
 - Đối với *consecutive_failed_logins* (*Số lần đăng nhập sai liên tiếp*): Phân hoạch dựa trên điều kiện kích hoạt trạng thái bị khóa của hệ thống. Vì đây là kiểm thử hộp đen thuần túy ở đầu vào (không can thiệp sửa trực tiếp DB), nên số lần sai ban đầu chỉ có thể là các số nguyên phi âm hợp lệ từ bên ngoài: nằm trong khoảng đăng nhập bình thường **[0, 2]** (Valid) và đạt tới ngưỡng khóa **>= 3** (Invalid).
 - Đối với *lockout_state* (*Trạng thái khóa và thời gian chờ*): Phân hoạch dựa trên việc tài khoản có đang trong thời gian phạt tạm khóa (đang khóa trong khoảng 30s) hay đang hoạt động bình thường.
 - Đối với *biến đầu ra* (*Outputs*): Phân hoạch dựa trên mã HTTP trả về, cấu trúc dữ liệu phản hồi (JSON chứa token/lỗi), và hành vi giao diện UI (hiển thị lỗi, khóa form đăng nhập hoặc chuyển hướng trang).
3. **Lựa chọn giá trị đại diện (Representatives):** Với mỗi lớp tương đương được phân hoạch, em chọn ra một giá trị đại diện cụ thể (ví dụ: **test@eshop.com** cho email tồn tại, **WrongPassword!** cho mật khẩu sai, số lần sai = 1 cho trường hợp bình thường).
4. **Thiết kế tập Test Cases tối thiểu:** Áp dụng nguyên tắc kết hợp các lớp tương đương:
 - Đăng nhập thành công (TC01)*: Kết hợp tất cả các lớp tương đương hợp lệ (**Valid**) của mọi biến đầu vào để kiểm tra luồng chính của hệ thống trong 1 test case duy nhất.
 - Kiểm thử các lớp lỗi (TC02 đến TC07)*: Đối với các lớp không hợp lệ (**Invalid**), em thiết kế mỗi test case chỉ chứa duy nhất **một** lớp không hợp lệ kết hợp với các lớp hợp lệ khác. Nguyên tắc này giúp cô lập lỗi (isolation) và tránh hiện tượng che giấu lỗi (error masking).
 - Các kịch bản tích hợp & bảo mật nâng cao (TC08 đến TC10)*: Bổ sung các kịch bản kiểm thử API trực tiếp, race condition concurrent và phiên làm việc để tăng độ bao phủ kiểm thử.

1. Các biến đầu vào (Input Variables)

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC01	email	Valid	Định dạng email hợp lệ và tồn tại trong CSDL	Đăng nhập bằng tài khoản có thực (ví dụ: test@eshop.com).
EC02	email	Invalid	Định dạng email hợp lệ nhưng không tồn tại trong CSDL	Đăng nhập bằng tài khoản không tồn tại (ví dụ: nonexistent@eshop.com).
EC03	email	Invalid	Định dạng email không hợp lệ	Nhập thiếu ký tự @, thiếu tên miền (ví dụ: invalid-email.com).
EC04	email	Invalid	Chuỗi rỗng	Bỏ trống email.
EC05	password	Valid	Trùng khớp với mật khẩu được lưu trong CSDL của email đó	Mật khẩu chính xác.
EC06	password	Invalid	Không trùng khớp với mật khẩu được lưu trong CSDL của email đó	Mật khẩu không chính xác.
EC07	password	Invalid	Chuỗi rỗng	Bỏ trống mật khẩu.
EC08	consecutive_failed_logins	Valid	Số nguyên nằm trong khoảng [0, 2]	Tài khoản đang hoạt động bình thường, chưa bị khóa.
EC09	consecutive_failed_logins	Invalid	Số nguyên >= 3	Tài khoản đang ở trạng thái bị tạm khóa do trước đó đã sai liên tiếp từ 3 lần trở lên.
EC10	lockout_state	Valid	Tài khoản không bị khóa (Active) hoặc thời gian khóa đã hết (t > 30 giây)	Người dùng có thể tiến hành đăng nhập bình thường.
EC11	lockout_state	Invalid	Tài khoản đang bị khóa và thời gian trôi qua 0 <= t <= 30 giây	Tài khoản đang trong thời gian phạt khóa 30 giây, mọi thao tác login đều bị chặn.

2. Các biến đầu ra (Output Variables)

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC12	http_status_code	Valid (Success)	200 OK	Đăng nhập thành công.
EC13	http_status_code	Invalid (Failure)	400 Bad Request hoặc 401 Unauthorized	Đăng nhập thất bại do sai tài khoản/mật khẩu hoặc lỗi định dạng.
EC14	http_status_code	Invalid (Failure)	403 Forbidden	Đăng nhập thất bại do tài khoản đang bị tạm khóa.
EC15	api_response_payload	Valid (Success)	JSON chứa JWT token và thông tin user (id, name, email, role)	Trả về thông tin đăng nhập thành công.
EC16	api_response_payload	Invalid (Failure)	JSON chứa thông báo lỗi bảo mật chung	Phản hồi lỗi không tiết lộ chi tiết nguyên nhân đăng nhập sai.
EC17	ui_message	Valid (Success)	Trống (không hiển thị lỗi)	Giao diện ở trạng thái bình thường.
EC18	ui_message	Invalid (Failure)	Chuỗi thông báo lỗi hiển thị trên nút submit	Giao diện hiển thị thông báo lỗi tương ứng với hành động thất bại.
EC19	ui_action	Valid (Success)	Redirect sang trang chủ	Client lưu token và thực hiện điều hướng trang.
EC20	ui_action	Invalid (Failure)	Giữ nguyên màn hình đăng nhập	Cho phép người dùng nhập lại thông tin.
EC21	ui_action	Invalid (Failure)	Lock form đăng nhập và đếm ngược 30 giây	Vô hiệu hóa nút Đăng nhập và bắt đầu đếm ngược thời gian khóa.

Bước 3: Lựa chọn giá trị đại diện (Selecting Representatives)

1. Bảng giá trị đại diện cho các lớp tương đương

Mã lớp	Biến tương ứng	Loại lớp	Giá trị đại diện	Ý nghĩa / Ghi chú kiểm thử
EC01	email	Valid	test@eshop.com	Tài khoản tồn tại trong hệ thống.
EC02	email	Invalid	nonexistent@eshop.com	Tài khoản không tồn tại.
EC03	email	Invalid	invalid-email.com	Sai định dạng email.
EC04	email	Invalid	"" (Chuỗi rỗng)	Bỏ trống email.
EC05	password	Valid	Test1234!	Mật khẩu trùng khớp.
EC06	password	Invalid	WrongPassword!	Mật khẩu không trùng khớp.
EC07	password	Invalid	"" (Chuỗi rỗng)	Bỏ trống mật khẩu.
EC08	consecutive_failed_logins	Valid	1	Số lần sai nằm trong khoảng hợp lệ trước khi khóa.
EC09	consecutive_failed_logins	Invalid	3	Số lần đăng nhập sai vượt ngưỡng cho phép (tài khoản đã bị khóa).
EC10	lockout_state	Valid	Không khóa (Active)	Tài khoản ở trạng thái bình thường.
EC11	lockout_state	Invalid	Đang bị khóa (Locked, ví dụ: vừa khóa được 15 giây)	Tài khoản đang bị tạm khóa.

2. Thiết kế tập Test Cases phân hoạch tương đương (Equivalence Partitioning Test Cases)

Tập test cases tối thiểu dưới đây được thiết kế nhằm bao phủ toàn bộ các lớp tương đương đã phân hoạch ở Bước 2 (áp dụng nguyên tắc kết hợp các lớp Valid và kiểm thử riêng lẻ từng lớp Invalid):

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thái (Pass/Fail)
TC01	Đăng nhập thành công	EC01, EC05, EC08, EC10, EC12, EC15, EC17, EC19	Tài khoản đã đăng nhập sai 1 lần trước đó	test@eshop.com	Test1234!	- HTTP Code: 200 OK - Response: JSON chứa JWT token & thông tin user. - UI / Client: Lưu	- HTTP Code: 200 OK - Response: Trả về thành công JWT token và user, tuy nhiên đối tượng user	Fail

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thái (Pass/Fail)
						trở thành công JWT Token phía client để sử dụng cho các yêu cầu có xác thực.	bị lộ mật khẩu chưa mã hóa "password": "Test1234!" và các trường nội bộ. - UI / Client: Lưu token thành công phía client.	
TC02	Đăng nhập thất bại do email không tồn tại	EC02, EC13, EC16, EC18, EC20	Không có (tài khoản chưa tồn tại)	nonexistent@eshop.com	Test1234!	- HTTP Code: 401 Unauthorized - Response: JSON chứa thông báo lỗi bảo mật chung, không phân biệt nguyên nhân. - UI: Hiện thị thông báo lỗi chung (không tiết lộ chi tiết nguyên nhân lỗi như "tài khoản không tồn tại" để tránh dò tài khoản).	- HTTP Code: 401 Unauthorized - Response: {"error": "Invalid email or password"} - UI: Hiện thị lỗi đăng nhập thất bại.	Pass
TC03	Đăng nhập thất bại do sai định dạng email	EC03, EC13, EC16, EC18, EC20	Không có	invalid-email.com	Test1234!	- UI: Trình duyệt chặn submit ngay tại client do HTML5 validation (yêu cầu nhập đúng định dạng email), không gửi request lên server.	- UI: Không chặn submit tại client, gửi request thành công lên server. - HTTP Code: 401 Unauthorized - Response: {"error": "Invalid email or password"}	Fail
TC04	Đăng nhập thất bại do bỏ trống email	EC04, EC13, EC16, EC18, EC20	Không có	""	Test1234!	- UI: Trình duyệt chặn submit ngay tại client do thuộc tính required (hiển thị tooltip yêu cầu nhập trường này), không gửi request lên server.	- UI: Trình duyệt chặn submit thành công, hiển thị tooltip thông báo lỗi "Please fill out this field". Không gửi request lên server.	Pass
TC05	Đăng nhập thất bại do sai mật khẩu	EC06, EC13, EC16, EC18, EC20	Tài khoản đã đăng nhập sai 1 lần trước đó	test@eshop.com	WrongPassword!	- HTTP Code: 401 Unauthorized - Response: JSON chứa thông báo lỗi bảo mật chung, không phân biệt nguyên nhân. - UI: Hiện thị thông báo lỗi chung (không	- HTTP Code: 401 Unauthorized - Response: {"error": "Invalid email or password"} - UI: Hiện thị lỗi đăng nhập thất bại.	Pass

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thái (Pass/Fail)
						tiết lộ chi tiết nguyên nhân lỗi như "sai mật khẩu").		
TC06	Đăng nhập thất bại do bỏ trống mật khẩu	EC07, EC13, EC16, EC18, EC20	Tài khoản hoạt động bình thường, chưa bị khóa	test@eshop.com	""	- UI: Trình duyệt chặn submit ngay tại client do thuộc tính required (hiển thị tooltip yêu cầu nhập trường này), không gửi request lên server.	- UI: Trình duyệt chặn submit thành công, hiển thị tooltip thông báo yêu cầu điền mật khẩu. Không gửi request lên server.	Pass
TC07	Đăng nhập khi số lần sai vượt ngưỡng (tài khoản khóa)	EC09, EC11, EC14, EC16, EC18, EC21	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó (đang trong thời gian tạm khóa 30 giây, ví dụ đã trôi qua 15 giây)	test@eshop.com	Test1234!	- HTTP Code: 403 Forbidden - Response: JSON chứa thông báo lỗi thích hợp về việc tài khoản bị khóa. - UI: Hiển thị thông báo lỗi thích hợp báo tài khoản bị khóa.	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."} - UI: Hiển thị thông báo lỗi báo tài khoản bị khóa.	Pass
TC08	Kiểm thử Brute-force song song (Race Condition)	EC09, EC14, EC21 (concurrency)	Tài khoản hoạt động bình thường, chưa từng đăng nhập sai	test@eshop.com	WrongPassword!	- Gửi đồng thời 5 request đăng nhập sai trong cùng 1 mili giây qua API. - Hệ thống phải áp dụng đúng ngưỡng khóa tài khoản và trả về 403 Forbidden cho các request vượt ngưỡng.	- Kết quả nhận được: Cả 5 request đều lọt qua bộ kiểm tra và trả về 401 Unauthorized cùng lúc, không request nào bị chặn 403 Forbidden trong loạt gửi song song. - Tài khoản chỉ bị khóa sau khi toàn bộ loạt request này thực thi xong.	Fail
TC09	Kiểm tra JWT token cũ sau khi tài khoản bị khóa ở session khác	EC09, EC11, EC14 (token check)	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó trên thiết bị khác (đang bị tạm khóa)	test@eshop.com	(Sử dụng JWT Token cũ)	Dùng token cũ của Thiết bị A gọi API /api/users/me sau khi Thiết bị B đã khóa tài khoản. Hệ thống trả về 401 Unauthorized hoặc 403 Forbidden.	- HTTP Code: 200 OK - Response: Trả về thành công thông tin user cá nhân chứa mật khẩu gốc lộ diện (password: "Test1234!"), không vô hiệu hóa token cũ của tài khoản bị khóa.	Fail
TC10	Gửi định dạng email sai trực tiếp qua	EC03, EC13, EC16	Không có	notanemail	Test1234!	Dùng Postman/cURL gửi trực tiếp request POST đến	- HTTP Code: 401 Unauthorized - Response: {"error":	Fail

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thái (Pass/Fail)
	Backend API					/api/login vượt qua Frontend. Backend trả về 400 Bad Request hoặc 401 Unauthorized kèm lỗi định dạng email.	"Invalid email or password"}. API trả lỗi đăng nhập chung, không có phản hồi validation riêng cho định dạng email sai.	

Bước 4: Phân tích giá trị biên (Boundary Value Analysis - BVA)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các giá trị biên nhảy cảm của tính năng **FR-02: Đăng nhập & Khóa tài khoản**, em đã thực hiện phân tích theo các bước sau:

1. **Xác định các biến có tính thứ tự hoặc khoảng số:** Trong các biến đầu vào đã xác định ở Bước 1, có 2 biến dạng số/thời gian liên tục là số lần đăng nhập sai liên tiếp (`consecutive_failed_logins`) và thời gian khóa tài khoản t (tính bằng giây).
2. **Xác định các điểm biên (Boundaries) cho từng biến:**
 - Số lần đăng nhập sai:* Nghiệp vụ quy định tài khoản bị khóa nếu đăng nhập sai từ 3 lần trở lên. Điều này có nghĩa là khoảng giá trị cho phép người dùng đăng nhập bình thường là $[0, 2]$. Do đó, biên dưới là $LB = 0$ (chưa sai lần nào) và biên trên là $UB = 2$ (số lần sai tối đa trước khi bị khóa ở lần tiếp theo).
 - Thời gian khóa tài khoản:* Nghiệp vụ quy định tài khoản bị tạm khóa trong 30 giây. Điều này có nghĩa là khoảng thời gian phạt khóa tài khoản là $[0, 30]$ giây. Do đó, biên dưới là $LB = 0s$ (thời điểm vừa bị khóa) và biên trên là $UB = 30s$ (giây cuối cùng trước khi hết hạn khóa).
3. **Lựa chọn các điểm kiểm thử biên nhảy cảm:** Với mỗi khoảng giá trị, em áp dụng nguyên tắc kiểm thử biên tiêu chuẩn gồm LB (biên dưới), UB (biên trên), các giá trị ngay sát ngoài biên dưới ($LB - 1$), sát ngoài biên trên ($UB + 1$), sát trong biên dưới ($LB + 1$) và sát trong biên trên ($UB - 1$) để tạo ra các test cases kiểm thử biên tương ứng. *Lưu ý: Các giá trị biên không thể kích hoạt từ các giao thức đầu vào công khai như bộ đếm âm ($LB - 1 = -1$) hay thời gian âm ($LB - 1 = -1s$) chỉ được ghi nhận dưới góc độ phân tích lý thuyết, nhưng không thiết kế test case trong bảng kiểm thử hộp đen thực tế do nguyên tắc không can thiệp chỉnh sửa trực tiếp Database.*

1. Phân tích giá trị biên của các biến số/khoảng số

- Biến `consecutive_failed_logins` (Khoảng hợp lệ trước khi bị khóa: $[0, 2]$):**
 - $LB = 0$: Số lần đăng nhập sai tối thiểu (chưa từng sai).
 - $LB + 1 = 1$: Đăng nhập sai 1 lần.
 - $UB = 2$: Số lần đăng nhập sai tối đa trước khi bị khóa ở lần tiếp theo.
 - $UB + 1 = 3$: Đăng nhập sai 3 lần, tài khoản chính thức bị khóa.
 - $LB - 1 = -1$: Giá trị âm không hợp lệ.
- Biến thời gian khóa t (giây) (Khoảng thời gian bị khóa: $[0, 30]$):**
 - $LB = 0s$: Vừa mới bắt đầu bị khóa.
 - $LB + 1 = 1s$: Đang bị khóa (thời gian trôi qua cực tiểu).
 - $UB = 30s$: Thời điểm kết thúc khóa 30 giây.
 - $UB + 1 = 31s$: Vừa hết thời gian khóa (tài khoản tự động mở khóa).
 - $LB - 1 = -1s$: Giá trị thời gian âm không hợp lệ.

2. Thiết kế tập Test Cases giá trị biên (Boundary Value Test Cases)

Để kiểm chứng tính đúng đắn của hệ thống tại các điểm biên nhảy cảm này, ta thực hiện các kịch bản kiểm thử sau:

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thi (Pass/Fai
TC-BVA-01	Đăng nhập khi chưa từng sai lần nào	Biên dưới $LB = 0$ của <code>consecutive_failed_logins</code>	Tài khoản chưa từng đăng nhập sai trước đó	test@eshop.com	Test1234!	- HTTP Code: 200 OK - Response: JSON chứa JWT token & thông tin user. - UI / Client: Lưu trữ thành công JWT Token phía client để sử	- HTTP Code: 200 OK - Response: Đăng nhập thành công, trả về JWT token và thông tin user. - UI / Client: Lưu token	Pass

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thái (Pass/Fail)
						dùng cho các yêu cầu có xác thực.	thành công phía client.	
TC-BVA-02	Đăng nhập sai lần đầu tiên	Biên dưới $LB = 0$ của <code>consecutive_failed_logins</code> với mật khẩu sai	Tài khoản chưa từng đăng nhập sai trước đó	test@eshop.com	WrongPassword!	- HTTP Code: 401 Unauthorized - Response: JSON chứa thông báo lỗi bảo mật chung.	- HTTP Code: 401 Unauthorized - Response: {"error": "Invalid email or password"}.	Pass
TC-BVA-03	Đăng nhập sai lần thứ 3 (trực tiếp gây khóa)	Biên trên $UB = 2$ của <code>consecutive_failed_logins</code> với mật khẩu sai	Tài khoản đã đăng nhập sai 2 lần liên tiếp trước đó	test@eshop.com	WrongPassword!	- HTTP Code: 401 Unauthorized - Response: JSON chứa thông báo lỗi bảo mật chung. - Hệ thống tự động chuyển sang trạng thái bị tạm khóa (các lần đăng nhập tiếp theo trong vòng 30s sẽ bị từ chối với mã 403).	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}. Tài khoản đã bị khóa từ trước đó (bị khóa sớm sau lần sai thứ 2) nên request này bị chặn ngay lập tức.	Fail
TC-BVA-04	Đăng nhập ngay khi vừa bị khóa	Biên dưới $LB = 0s$ của thời gian khóa t	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó (vừa mới bị khóa, $t = 0$ giây)	test@eshop.com	Test1234!	- HTTP Code: 403 Forbidden - Response: JSON chứa thông báo lỗi thích hợp về việc tài khoản bị khóa. - UI: Hiện thị thông báo lỗi thích hợp báo tài khoản bị khóa.	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}. - UI / Thực tế: Nhập sai 3 lần (lần 3 bị chặn 403 do khóa từ lần 2), đăng nhập đúng ngay sau đó bị chặn thành công bằng mã 403 ở giây thứ 0.	Pass
TC-BVA-05	Đăng nhập khi đang bị khóa	Giá trị lân cận biên dưới $LB + 1 = 1s$ của thời gian khóa t	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó (đang trong thời gian khóa, $t = 1$ giây)	test@eshop.com	Test1234!	- HTTP Code: 403 Forbidden - Response: JSON chứa thông báo lỗi thích hợp về việc tài khoản bị khóa. - UI: Hiện thị thông báo lỗi thích hợp báo tài khoản bị khóa.	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}. - UI / Thực tế: Đăng nhập đúng ở giây thứ 1 bị chặn thành công bằng mã 403.	Pass

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	email	password	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)	Trạng thi (Pass/Fai
TC-BVA-06	Đăng nhập tại thời điểm giây thứ 30 của khóa	Biên trên $UB = 30s$ của thời gian khóa t	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó (đang trong thời gian khóa, $t = 30$ giây)	test@eshop.com	Test1234!	- HTTP Code: 403 Forbidden - Response: JSON chứa thông báo lỗi thích hợp về việc tài khoản bị khóa. - UI: Hiện thị thông báo lỗi thích hợp báo tài khoản bị khóa.	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}. - UI / Thực tế: Đăng nhập đúng ở giây thứ 30 bị chặn thành công bằng mã 403 (tài khoản vẫn đang bị khóa).	Pass
TC-BVA-07	Đăng nhập thành công ngay khi vừa hết hạn khóa	Giá trị lân cận biên trên $UB + 1 = 31s$ của thời gian khóa t	Tài khoản đã đăng nhập sai 3 lần liên tiếp trước đó (vừa hết thời gian tạm khóa 30 giây, $t = 31$ giây)	test@eshop.com	Test1234!	- HTTP Code: 200 OK - Response: JSON chứa JWT token & thông tin user. - UI / Client: Lưu trữ thành công JWT Token phía client để sử dụng cho các yêu cầu có xác thực.	- HTTP Code: 403 Forbidden - Response: {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."}. - UI / Thực tế: Đăng nhập đúng tại giây thứ 31 bị từ chối bằng mã 403, tài khoản vẫn tiếp tục bị khóa (không tự động mở khóa sau 30 giây như đặc tả).	Fail

Bước 5: Phân tích khoảng trống AI (AI Gap Analysis)

1. Các kịch bản/lỗi kiểm thử mà AI đã bỏ sót

- **Đồng thời brute-force (Race Condition):** AI bỏ sót kịch bản người dùng gửi liên tiếp nhiều request đăng nhập sai trong cùng một thời điểm rất ngắn (concurrency). Từ góc nhìn hộp đen, cần quan sát xem API có chặn đúng theo ngưỡng khóa hay không khi nhiều request xảy ra gần như đồng thời.
- **Vô hiệu hóa phiên làm việc (Session/Token Invalidation):** AI chưa đề xuất kiểm thử xem JWT token cũ đã được cấp tử trước có bị vô hiệu hóa ngay lập tức khi tài khoản bị khóa ở một phiên khác hay không.

2. Các sự nhầm lẫn, ảo giác và thiếu sót của AI trong quá trình thiết kế (AI Critique)

- **Nhầm lẫn giữa Kiểm thử Hộp đen (Black-box) và Hộp xám/Hộp trắng (Grey-box/White-box):**
 - *Ảo giác kiểm tra trực tiếp Database ở Expected Output:* Trong các phiên thảo luận ban đầu, AI liên tục đưa các câu lệnh kiểm tra dữ liệu trực tiếp trong Database (ví dụ: yêu cầu kiểm tra trường login_attempts trong database sau khi test case chạy xong) vào cột **Kết quả mong đợi (Expected Output)**. Điều này vi phạm nguyên lý hộp đen vì các trường này nằm ẩn trong CSDL và chỉ có thể được kiểm chứng gián tiếp thông qua các hành vi giao diện hoặc API bên ngoài.
 - *Đề xuất test case can thiệp DB trực tiếp:* AI đề xuất các test case âm phi thực tế như thiết lập bộ đếm sai thành -1 hay "two" trong database trước khi chạy test, vốn không thể thực hiện được thông qua các giao diện UI/API công khai của người dùng cuối.
- **Nhầm lẫn về vai trò của biến đầu vào (Inputs) và Trạng thái hệ thống (Preconditions):**
 - AI sử dụng các tên biến kỹ thuật tương tự tên cột trong Database như failed_login_attempts và lockout_status rồi xếp chúng vào cột biến đầu vào (Input Variables). Việc này gây hiểu nhầm rằng tester có thể nhập các giá trị này trực tiếp từ form đăng nhập. Trên thực tế, chúng là **Trạng thái hệ thống (System State)**, cần được mô hình hóa dưới dạng **Điều kiện tiên đề (Preconditions)**.
- **Xu hướng bị ảnh hưởng bởi code cài đặt thực tế (Implementation Bias):**
 - AI đưa chính xác các chuỗi JSON lỗi cụ thể của backend như {"error": "Invalid email or password"} hay {"error": "Tài khoản đã bị khóa. Vui lòng thử lại sau."} vào cột Kết quả mong đợi (Expected Output) thay vì mô tả yêu cầu nghiệp vụ chung từ đặc tả.

3. Giải thích nguyên nhân AI gặp các hạn chế trên

- Hạn chế của công cụ AI (Limitations of the AI tool itself):** AI thực hiện kiểm thử hộp đen tĩnh dựa trên văn bản đặc tả. Nó không tự chạy mã nguồn SUT hoặc mô phỏng môi trường động. Vì vậy, các khía cạnh kỹ thuật như xử lý bất đồng bộ (concurrency), tranh chấp tài nguyên (race conditions), hay các lỗi đồng bộ bảo mật nâng cao nằm ngoài khả năng suy luận mặc định của AI.
- Độ phức tạp nội tại của tính năng (Inherent complexity of the feature under test):** FR-02 nhìn bên ngoài giao diện rất đơn giản (chỉ có form Email và Password), nhưng phía sau backend là sự kết hợp phức tạp giữa quản lý trạng thái và cơ chế xác thực không lưu trạng thái (stateless JWT token). AI có xu hướng tập trung vào các luồng chức năng bề nổi (functional UI flows) mà bỏ qua các logic bảo mật phi chức năng (non-functional security logic).
- Chất lượng của dữ liệu đầu vào (Prompt Quality):** Các prompt ban đầu chỉ yêu cầu AI đọc tài liệu đặc tả chung mà chưa cung cấp ngữ cảnh về môi trường triển khai thực tế, các yêu cầu kiểm thử phi chức năng hay các tiêu chuẩn an toàn thông tin (như OWASP). Điều này khiến AI chỉ tập trung tối ưu hóa các phân vùng giá trị biên của email/password theo nghiệp vụ thông thường mà bỏ qua các trường hợp biên của bảo mật hệ thống.

Pool B: FR-09: Mã Giảm Giá (Coupon)

Bước 1: Xác định các biến Input và Output (I/O Variables)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các biến vào/ra của tính năng **FR-09: Mã Giảm Giá (Coupon)**, em đã thực hiện các bước phân tích sau:

- Phân tích đặc tả nghiệp vụ (Specification Analysis):** Đọc kỹ tài liệu đặc tả yêu cầu hệ thống **README.md**. Chức năng này yêu cầu hệ thống áp dụng mã giảm giá khi người dùng nhập mã tại bước Checkout dựa trên 5 điều kiện (C1 đến C5) đồng thời thỏa mãn. Có 2 loại giảm giá là theo phần trăm (**percent**) và cố định (**fixed**).
- Xác định biến đầu vào trực tiếp (Direct Inputs):** Các tham số người dùng nhập trực tiếp hoặc gửi qua API body của endpoint **POST /api/apply-coupon** bao gồm: mã giảm giá (**code**), tổng số tiền đơn hàng gốc (**total_amount**), ID người dùng áp dụng mã (**user_id**), và Token JWT xác thực trong header (**jwt_token / Authorization**).
- Xác định biến đầu vào trạng thái (System State Inputs):** Trạng thái của mã giảm giá trong hệ thống bao gồm hoạt động hay không (**coupon_state**), ngày hết hạn (**coupon_expiration**), và lịch sử sử dụng của người dùng (**user_coupon_usage**). Các biến này không thể nhập trực tiếp qua form UI của client mà được lưu trữ trong CSDL và dùng làm **Điều kiện tiên đề (Preconditions)** cho các kịch bản kiểm thử.
- Xác định biến đầu ra (Outputs):** Ở mức API, backend trả về mã trạng thái HTTP (**http_status_code**), số tiền được giảm (**discount_amount**), và số tiền cuối cùng phải trả (**final_amount**). Ở mức UI, giao diện hiển thị thông báo kết quả (**ui_message**) và thực hiện hành động cập nhật giá trị hiển thị hoặc báo lỗi (**ui_action**).

1. Các biến đầu vào (Input Variables)

Dưới đây là danh sách các biến đầu vào của chức năng:

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	code	Direct Input	String	Ký tự chữ và số, không chứa ký tự đặc biệt, không trống	Mã giảm giá do người dùng nhập vào.
2	total_amount	Direct Input	Integer	Số nguyên dương >= 0	Tổng tiền đơn hàng trước khi giảm giá.
3	user_id	Direct Input	Integer	Số nguyên dương > 0	ID của người dùng áp dụng mã giảm giá.
4	jwt_token	Direct Input	String	Định dạng chuỗi JWT hợp lệ	Token xác thực truyền qua header Authorization .
5	coupon_state	State Input	Enum	Active (is_active = 1) / Inactive (is_active = 0) / Không tồn tại	Trạng thái hoạt động của mã trong DB — Dùng làm Precondition .
6	coupon_expiration	State Input	DateTime	Ngày cụ thể (expired_at)	Hạn sử dụng của mã — Dùng làm Precondition .
7	user_coupon_usage	State Input	Integer	Số nguyên không âm (>= 0)	Số lần người dùng đã sử dụng mã này — Dùng làm Precondition .

2. Các biến đầu ra (Output Variables)

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	http_status_code	API Output	Integer	200 (Thành công) / 400, 401, 403, 404 (Thất bại)	Mã phản hồi HTTP từ server.
2	discount_amount	API Output	Integer	Số nguyên không âm (>= 0)	Số tiền được giảm giá dựa trên công thức tính.
3	final_amount	API Output	Integer	Số nguyên không âm (>= 0)	Số tiền cuối cùng sau giảm giá.
4	ui_message	UI Output	String	Thông báo thành công hoặc thông báo lỗi nghiệp vụ chi tiết	Cảnh báo hoặc thông điệp phản hồi hiển thị cho người dùng.

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
5	ui_action	UI Output	Enum	Render giá mới / Hiển thị lỗi và giữ nguyên giá cũ	Phản ứng hành vi của giao diện thanh toán.

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để thực hiện phân hoạch tương đương cho tính năng **FR-09: Mã Giảm Giá (Coupon)**, em đã áp dụng quy trình sau:

1. **Xác định các điều kiện nghiệp vụ:** Phân tích 5 điều kiện (C1 đến C5) cùng với 2 loại coupon (phần trăm và cố định).
2. **Chia nhóm các lớp tương đương:** Mỗi điều kiện ràng buộc được chia thành một lớp hợp lệ (Valid EC) đại diện cho việc thỏa mãn điều kiện và các lớp không hợp lệ (Invalid EC) đại diện cho các cách vi phạm khác nhau.
3. **Đánh số lại các lớp tương đương (EC):** Với mỗi tính năng, mã lớp tương đương bắt đầu lại từ **EC01**. Cụ thể, các lớp đầu vào của FR-09 được đánh số từ **EC01** đến **EC20**, và các lớp đầu ra được đánh số từ **EC21** đến **EC31**.
4. **Lựa chọn giá trị đại diện và thiết kế Test Cases:** Mỗi kịch bản chỉ kiểm tra một nguyên nhân lỗi duy nhất (một Invalid EC) kết hợp với các EC hợp lệ còn lại để tránh che giấu lỗi.

1. Các biến đầu vào (Input Variables)

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC01	code	Valid	Mã tồn tại trong hệ thống và đang hoạt động	Kiểm tra mã hợp lệ như SAVE10 , BIGBUY , VIP100 .
EC02	code	Invalid	Mã không tồn tại trong hệ thống	Nhập mã sai chính tả hoặc ngẫu nhiên.
EC03	code	Invalid	Mã tồn tại nhưng ở trạng thái ngưng hoạt động	Mã có is_active = 0 trong database.
EC04	code	Invalid	Chuỗi rỗng	Bỏ trống không nhập mã giảm giá.
EC05	total_amount	Valid	Lớn hơn hoặc bằng ngưỡng tối thiểu (min_order_amount)	Thỏa mãn điều kiện C3 về giá trị đơn hàng.
EC06	total_amount	Invalid	Nhỏ hơn ngưỡng tối thiểu (min_order_amount)	Vi phạm điều kiện C3.
EC07	total_amount	Invalid	Số âm hoặc bằng 0	Giá trị đơn hàng không hợp lệ.
EC08	total_amount	Invalid	Không truyền hoặc truyền sai kiểu dữ liệu	Gửi giá trị không phải số nguyên.
EC09	user_id	Valid	Trùng khớp với người dùng đang đăng nhập và có thực	Áp dụng mã cho chính chủ tài khoản.
EC10	user_id	Invalid	Khác với người dùng được mã hóa trong JWT Token	Cố tình giả mạo hoặc áp dụng mã cho user khác.
EC11	user_id	Invalid	Không tồn tại trong CSDL hoặc không hợp lệ	user_id âm, bằng 0, hoặc quá lớn.
EC12	user_id	Invalid	Chuỗi rỗng hoặc không truyền	Thiếu trường thông tin bắt buộc.
EC13	jwt_token	Valid	Token hợp lệ, chưa hết hạn	Người dùng đã đăng nhập hợp lệ (C4).
EC14	jwt_token	Invalid	Token không hợp lệ, hết hạn hoặc không truyền	Người dùng chưa đăng nhập hoặc token giả.
EC15	coupon_state	Valid	Trạng thái đang hoạt động (is_active = 1)	Đáp ứng điều kiện C1 về trạng thái coupon.
EC16	coupon_state	Invalid	Trạng thái bị vô hiệu hóa (is_active = 0)	Coupon bị tạm khóa/ngưng hoạt động bởi admin.
EC17	coupon_expiration	Valid	Ngày hiện tại nhỏ hơn ngày hết hạn (expired_at)	Coupon vẫn còn trong hạn sử dụng (C2).
EC18	coupon_expiration	Invalid	Ngày hiện tại lớn hơn hoặc bằng ngày hết hạn (expired_at)	Coupon đã hết hạn sử dụng.
EC19	user_coupon_usage	Valid	Số lần đã dùng < giới hạn tối đa (max_uses_per_user)	Người dùng còn lượt sử dụng coupon này (C5).
EC20	user_coupon_usage	Invalid	Số lần đã dùng >= giới hạn tối đa (max_uses_per_user)	Người dùng đã dùng hết số lần tối đa cho phép.

2. Các biến đầu ra (Output Variables)

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC21	http_status_code	Valid (Success)	200 OK	Áp dụng mã giảm giá thành công.

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC22	http_status_code	Invalid (Failure)	400 Bad Request	Lỗi nghiệp vụ (hết hạn, thiếu tiền tối thiểu, hết lượt dùng) hoặc sai định dạng.
EC23	http_status_code	Invalid (Failure)	401 Unauthorized	Không có quyền hoặc token hết hạn/không hợp lệ.
EC24	http_status_code	Invalid (Failure)	403 Forbidden	Từ chối áp dụng mã do sai lệch ID người dùng.
EC25	http_status_code	Invalid (Failure)	404 Not Found	Không tìm thấy mã giảm giá trong hệ thống.
EC26	discount_amount & final_amount	Valid (Success)	Số tiền giảm và số tiền cuối tính chính xác	Áp dụng đúng công thức chiết khấu phần trăm hoặc giá cố định.
EC27	discount_amount & final_amount	Invalid (Failure)	discount_amount = 0, final_amount giữ nguyên giá trị đơn hàng	Không thực hiện giảm trừ tiền.
EC28	ui_message	Valid (Success)	Hiển thị thông báo áp dụng mã thành công	Xác nhận mã giảm giá hợp lệ.
EC29	ui_message	Invalid (Failure)	Hiển thị thông báo lỗi chi tiết tương ứng	Báo lỗi đúng nguyên nhân (ví dụ: mã hết hạn, chưa đủ ngưỡng tối thiểu,...).
EC30	ui_action	Valid (Success)	Cập nhật số tiền hiển thị trên giao diện thanh toán	Giao diện hiển thị giá sau giảm.
EC31	ui_action	Invalid (Failure)	Hiển thị thông báo lỗi, giữ nguyên giá cũ	Chặn việc áp dụng giảm giá trên UI.

Bước 3: Lựa chọn giá trị đại diện (Selecting Representatives)

1. Bảng giá trị đại diện cho các lớp tương đương

Mã lớp	Biến tương ứng	Loại lớp	Giá trị đại diện	Ý nghĩa / Ghi chú kiểm thử
EC01	code	Valid	SAVE10	Mã giảm giá đang hoạt động và hợp lệ trong CSDL.
EC02	code	Invalid	NOTFOUND	Mã giảm giá không tồn tại trong hệ thống.
EC03	code	Invalid	SAVE10 (nhưng DB có is_active = 0)	Mã giảm giá bị ngưng hoạt động.
EC04	code	Invalid	""	Bỏ trống mã giảm giá.
EC05	total_amount	Valid	500000	Tổng tiền lớn hơn ngưỡng tối thiểu 300000 của mã SAVE10.
EC06	total_amount	Invalid	200000	Tổng tiền nhỏ hơn ngưỡng tối thiểu 300000 của mã SAVE10.
EC07	total_amount	Invalid	-50000	Số tiền âm không hợp lệ.
EC08	total_amount	Invalid	"five_hundred" (hoặc bỏ trống)	Sai kiểu dữ liệu truyền lên.
EC09	user_id	Valid	1	ID của người dùng thực hiện yêu cầu (khớp với JWT token).
EC10	user_id	Invalid	2	ID người dùng khác với thông tin trong token của user 1.
EC11	user_id	Invalid	9999	ID người dùng không tồn tại trong hệ thống.
EC12	user_id	Invalid	"" (hoặc bỏ trống)	Bỏ trống trường user_id.
EC13	jwt_token	Valid	Token hợp lệ của User 1	Người dùng đã đăng nhập.
EC14	jwt_token	Invalid	Token sai/hết hạn hoặc thiếu header	Người dùng chưa đăng nhập hoặc phiên làm việc hết hạn.
EC15	coupon_state	Valid	is_active = 1	Mã giảm giá đang kích hoạt.
EC16	coupon_state	Invalid	is_active = 0	Mã giảm giá bị vô hiệu hóa.
EC17	coupon_expiration	Valid	Hạn dùng 2099-12-31	Mã giảm giá còn hạn sử dụng.
EC18	coupon_expiration	Invalid	EXPIRED (hạn dùng 2020-01-01)	Mã giảm giá đã hết hạn sử dụng.
EC19	user_coupon_usage	Valid	Đã dùng 0 lần	Còn lượt sử dụng (giới hạn tối đa là 1).
EC20	user_coupon_usage	Invalid	Đã dùng 1 lần	Hết lượt sử dụng (giới hạn tối đa là 1).

2. Thiết kế tập Test Cases phân hoạch tương đương (Equivalence Partitioning Test Cases)

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	code	total_amount	user_id	jwt_token	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)
TC01	Áp dụng thành công coupon loại percent	EC01, EC05, EC09, EC13, EC15, EC17, EC19, EC21, EC26, EC28, EC30	Mã SAVE10 đang hoạt động, còn hạn, giới hạn 1 lần/người. Người dùng chưa từng sử dụng mã này.	SAVE10	500000	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON chứa discount_amount = 50,000 và final_amount = 450,000. - UI: Hiển thị giá đã giảm và thông báo áp dụng thành công.	- HTTP Code: 200 - Response: JSON - UI: Hiển thị thông
TC02	Áp dụng thành công coupon loại fixed	EC01, EC05, EC09, EC13, EC15, EC17, EC19, EC21, EC26, EC28, EC30	Mã BIGBUY (giảm 50,000đ, tối thiểu 500,000đ) đang hoạt động, còn hạn. Người dùng chưa từng sử dụng mã này.	BIGBUY	600000	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON chứa discount_amount = 50,000 và final_amount = 550,000. - UI: Hiển thị giá đã giảm và thông báo áp dụng thành công.	- HTTP Code: 200 - Response: JSON - UI: Hiển thị thông
TC03	Áp dụng thất bại - Mã giảm giá không tồn tại	EC02, EC25, EC27, EC29, EC31	Người dùng đăng nhập bình thường.	NOTFOUND	500000	1	Token hợp lệ User 1	- HTTP Code: 404 Not Found - Response: JSON chứa thông báo lỗi không tìm thấy coupon. - UI: Hiển thị thông báo "Mã không tồn tại".	- HTTP Code: 404 - Response: JSON - UI: Hiển thị thông
TC04	Áp dụng thất bại - Mã giảm giá bị ngưng hoạt động	EC03, EC16, EC22, EC27, EC29, EC31	Mã SAVE10 đã bị chuyển trạng thái hoạt động sang is_active = 0 trong CSDL.	SAVE10	500000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request hoặc 404 Not Found - Response: JSON chứa thông báo lỗi phù hợp. - UI: Hiển thị thông báo lỗi.	- HTTP Code: 404 - Response: JSON - UI: Hiển thị thông
TC05	Áp dụng thất bại - Bỏ trống mã giảm giá	EC04, EC22, EC27, EC29, EC31	Người dùng đăng nhập bình thường.	""	500000	1	Token hợp lệ User 1	- UI: Hiển thị lỗi hoặc chặn gửi request. - HTTP Code (nếu gửi trực tiếp API): 400 Bad Request chứa thông báo mã giảm giá là bắt buộc.	- UI: Nút "Áp dụng" - HTTP Code (nếu
TC06	Áp dụng thất bại - Tổng tiền đơn hàng chưa đạt ngưỡng tối thiểu	EC06, EC22, EC27, EC29, EC31	Mã SAVE10 yêu cầu tối thiểu 300,000đ. Người dùng chưa dùng mã này.	SAVE10	200000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON chứa thông báo lỗi đơn hàng không đủ giá trị tối thiểu. - UI: Hiển thị thông báo "Giá trị đơn hàng tối thiểu chưa đạt".	- HTTP Code: 400 - Response: JSON - UI: Hiển thị thông
TC07	Áp dụng thất bại - Số	EC07, EC22, EC27, EC29,	Người dùng đăng nhập bình	SAVE10	-50000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request	- HTTP Code: 400 - Response: JSON

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	code	total_amount	user_id	jwt_token	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)
	tiền đơn hàng âm	EC31	thường.					- Response: JSON chứa thông báo lỗi số tiền không hợp lệ. - UI: Không cho phép gửi hoặc hiển thị lỗi số tiền không hợp lệ.	- UI: Hiển thị sai th
TC08	Áp dụng thất bại - Định dạng số tiền sai kiểu dữ liệu	EC08, EC22, EC27, EC29, EC31	Người dùng đăng nhập bình thường.	SAVE10	"five_hundred"	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request hoặc 422 Unprocessable Entity - Response: JSON chứa thông báo lỗi định dạng dữ liệu. - UI: Chặn gửi hoặc báo lỗi nhập liệu.	- HTTP Code: 400 - Response: {"err trực tiếp API). - UI: Trình duyệt m
TC09	Áp dụng thất bại - Người dùng giả mạo user_id trong body	EC10, EC24, EC27, EC29, EC31	Gửi trực tiếp qua backend API, thay thế user_id trong body khác với ID lưu trong JWT Token.	SAVE10	500000	2	Token hợp lệ User 1	- HTTP Code: 403 Forbidden - Response: JSON thông báo từ chối quyền truy cập hoặc lỗi bảo mật.	- HTTP Code: 200 - Response: {"success":true dụng thành công
TC10	Áp dụng thất bại - user_id không tồn tại	EC11, EC22, EC27, EC29, EC31	Người dùng đăng nhập bình thường nhưng truyền ID không có thực.	SAVE10	500000	9999	Token hợp lệ User 1	- HTTP Code: 400 Bad Request hoặc 404 Not Found - Response: JSON báo người dùng không tồn tại.	- HTTP Code: 200 - Response: {"success":true dụng thành công
TC11	Áp dụng thất bại - Chưa đăng nhập hoặc token hết hạn	EC14, EC23, EC27, EC29, EC31	Người dùng chưa đăng nhập hoặc token đã bị sửa đổi/hết hạn.	SAVE10	500000	1	Token không hợp lệ hoặc thiếu	- HTTP Code: 401 Unauthorized - Response: JSON chứa thông báo lỗi chưa xác thực.	- HTTP Code: 200 - Response: {"success":true dụng thành công
TC12	Áp dụng thất bại - Mã giảm giá hết hạn	EC18, EC22, EC27, EC29, EC31	Mã EXPIRED có hạn dùng 2020-01-01 (đã hết hạn so với hiện tại).	EXPIRED	200000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON chứa thông báo mã đã hết hạn. - UI: Hiển thị thông báo "Mã giảm giá đã hết hạn sử dụng".	- HTTP Code: 400 - Response: JSON - UI: Hiển thị thông
TC13	Áp dụng thất bại - Người dùng đã dùng hết lượt cho phép	EC20, EC22, EC27, EC29, EC31	Mã SAVE10 giới hạn 1 lần sử dụng/người. Người dùng 1 đã có 1 đơn hàng trước đó áp dụng mã này thành công.	SAVE10	500000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON chứa thông báo người dùng đã dùng hết lượt. - UI: Hiển thị thông báo "Bạn đã sử dụng hết lượt cho phép đối với mã này".	- HTTP Code: 400 - Response: JSON - UI: Hiển thị thông
TC14	Kiểm thử Race Condition - Đồng thời áp dụng mã	EC20, EC22, EC27 (concurrency)	Mã SAVE10 giới hạn 1 lần. Người dùng chưa từng sử dụng.	SAVE10	500000	1	Token hợp lệ User 1	- Gửi đồng thời 2 request áp dụng mã trong cùng 1ms. - Chỉ có đúng 1	- Cả 2 request gửi

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	code	total_amount	user_id	jwt_token	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)
	giới hạn 1 lần							request được chấp nhận (200 OK), request còn lại bị từ chối (400 Bad Request).	
TC15	Áp dụng mã thành công nhưng sửa đổi giỏ hàng khi checkout (Checkout Bypass)	EC01, EC06 (checkout validation)	Mã SAVE10 (ngưỡng 300,000đ) hoạt động, còn hạn. Người dùng thêm sản phẩm để tổng tiền = 500,000đ, áp dụng mã thành công, sau đó sửa đổi giỏ hàng giảm xuống còn 100,000đ và thực hiện Checkout.	SAVE10	100000 (tại checkout)	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON thông báo đơn hàng không đủ điều kiện tối thiểu để áp dụng mã. - Hệ thống không cho phép tạo đơn hàng thành công với giá đã giảm.	- HTTP Code: 200 - Response: JSON - UI: Thanh toán và
TC16	Áp dụng mã giảm giá lịch mức giờ client/server	EC18 (expiration check)	Mã EXPIRED đã hết hạn (2020-01-01). Người dùng cố tình sửa đổi mức giờ hoặc giờ hệ thống phía client về năm 2019 để gửi request.	EXPIRED	500000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON thông báo mã giảm giá đã hết hạn. - UI: Hiển thị lỗi mã giảm giá đã hết hạn sử dụng.	- HTTP Code: 400 - Response: JSON - UI: Hiển thị thông

Bước 4: Phân tích giá trị biên (Boundary Value Analysis - BVA)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các giá trị biên nhạy cảm của tính năng **FR-09: Mã Giảm Giá (Coupon)**, em đã thực hiện phân tích theo các bước sau:

1. Xác định các biến có tính thứ tự hoặc khoảng số:

 - `total_amount` (Tổng tiền đơn hàng): Có các ngưỡng biên nhạy cảm so với ngưỡng tối thiểu `min_order_amount` của từng mã giảm giá.
 - `user_coupon_usage` (Số lần người dùng đã sử dụng mã): Có giới hạn tối đa là `max_uses_per_user`.
2. Xác định các điểm biên (Boundaries) cho từng biến:

 - So với mã **SAVE10** có `min_order_amount = 300,000 VND`:
 - Biên dưới hợp lệ: $LB = 300,000 \text{ VND}$.
 - Sát trên biên dưới hợp lệ: $LB + 1 = 300,001 \text{ VND}$.
 - Sát dưới biên dưới không hợp lệ: $LB - 1 = 299,999 \text{ VND}$.
 - So với mã **VIP100** có `max_uses_per_user = 2`:
 - Số lần đã dùng tối đa hợp lệ để vẫn còn dùng được tiếp: $UB = 1$ lần (vì số lần dùng tiếp theo sẽ là lần thứ 2, đạt mức tối đa).
 - Số lần đã dùng bắt đầu không còn lượt: $UB + 1 = 2$ lần (đã dùng hết lượt, không thể dùng tiếp).
3. Thiết kế các kịch bản kiểm thử biên tương ứng.
1. Phân tích giá trị biên của các biến số/khoảng số

 - Biến **`total_amount`** so với ngưỡng tối thiểu `min_order_amount = 300,000 VND` của mã **SAVE10**:
 - $LB = 300,000 \text{ VND}$: Tổng tiền tối thiểu vừa đủ để áp dụng mã giảm giá.
 - $LB + 1 = 300,001 \text{ VND}$: Tổng tiền lớn hơn ngưỡng tối thiểu một đơn vị nhỏ nhất.
 - $LB - 1 = 299,999 \text{ VND}$: Tổng tiền nhỏ hơn ngưỡng tối thiểu một đơn vị nhỏ nhất (không đủ điều kiện).
 - Biến **`user_coupon_usage`** so với giới hạn `max_uses_per_user = 2` của mã **VIP100**:
 - $usage = 0$: Người dùng chưa từng sử dụng, còn nguyên 2 lượt.
 - $usage = 1$ (Điểm biên UB để còn lượt): Người dùng đã dùng 1 lần, vẫn còn 1 lượt nữa.
 - $usage = 2$ (Điểm biên $UB + 1$ để hết lượt): Người dùng đã dùng 2 lần, không còn lượt nào để sử dụng.
 - Biến **`coupon_expiration`** (Hạn sử dụng) của mã **SAVE10** (Hạn dùng **2099-12-31 23:59:59**):
 - $LB = 2099 - 12 - 31 \text{ 23} : 59 : 59$ (Điểm biên UB): Thời điểm cuối cùng hợp lệ để sử dụng mã.
 - $UB + 1 = 2100 - 01 - 01 \text{ 00} : 00 : 00$: Vừa hết hạn sử dụng 1 giây (không hợp lệ).

o $UB - 1 = 2099 - 12 - 31\ 23 : 59 : 58$: Còn hạn sử dụng 1 giây (hợp lệ).

2. Thiết kế tập Test Cases giá trị biên (Boundary Value Test Cases)

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	code	total_amount	user_id	jwt_token	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)
TC-BVA-01	Áp dụng khi tổng tiền bằng đúng ngưỡng tối thiểu	$total_amount = LB$ (300,000) của mã SAVE10	Mã SAVE10 hoạt động, còn hạn. Người dùng chưa từng dùng mã.	SAVE10	300000	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON chứa discount_amount = 30,000 và final_amount = 270,000. - UI: Hiển thị giá mới đã áp dụng.	- HTTP Code: 400 Bad Request - Response: JSON chứa error - UI: Hiển thị thông báo lỗi
TC-BVA-02	Áp dụng khi tổng tiền sát dưới ngưỡng tối thiểu	$total_amount = LB - 1$ (299,999) của mã SAVE10	Mã SAVE10 hoạt động, còn hạn. Người dùng chưa từng dùng mã.	SAVE10	299999	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON báo lỗi trị giá đơn hàng tối thiểu chưa đạt. - UI: Báo lỗi không đủ điều kiện đơn hàng tối thiểu.	- HTTP Code: 400 Bad Request - Response: JSON chứa error - UI: Hiển thị thông báo lỗi
TC-BVA-03	Áp dụng khi tổng tiền sát trên ngưỡng tối thiểu	$total_amount = LB + 1$ (300,001) của mã SAVE10	Mã SAVE10 hoạt động, còn hạn. Người dùng chưa từng dùng mã.	SAVE10	300001	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON chứa discount_amount = 30,000 và final_amount = 270,001. - UI: Hiển thị giá mới đã áp dụng.	- HTTP Code: 200 OK - Response: {"success":true,"code":200,"message":"Áp dụng thành công! Giảm 30,000 VND."} - UI: Hiển thị thông báo thành công
TC-BVA-04	Áp dụng mã khi người dùng đã sử dụng 1 lần (vẫn còn lượt)	$usage = 1$ của mã VIP100 (giới hạn tối đa 2 lần)	Mã VIP100 hoạt động, còn hạn. Người dùng đã từng dùng mã này 1 lần.	VIP100	500000	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON chứa discount_amount = 100,000 và final_amount = 400,000. - UI: Hiển thị giảm giá thành công lần 2.	- HTTP Code: 200 OK - Response: {"success":true,"code":200,"message":"Áp dụng thành công! Giảm 100,000 VND."} - UI: Hiển thị thông báo áp dụng thành công
TC-BVA-05	Áp dụng mã khi người dùng đã sử dụng 2 lần (hết lượt)	$usage = 2$ của mã VIP100 (giới hạn tối đa 2 lần)	Mã VIP100 hoạt động, còn hạn. Người dùng đã từng dùng mã này 2 lần.	VIP100	500000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON báo lỗi đã dùng hết số lần tối đa. - UI: Hiển thị lỗi đã hết lượt dùng.	- HTTP Code: 400 Bad Request - Response: {"error":true,"code":400,"message":"Bạn đã dùng hết lượt giảm giá."} - UI: Hiển thị thông báo lỗi
TC-BVA-06	Áp dụng mã giảm giá sát giờ hết hạn (còn hạn 1 giây)	$expired_at - 1s$ của mã SAVE10	Mã SAVE10 đang hoạt động. Thời gian hệ thống hiện tại là sát trước giờ hết hạn 1 giây.	SAVE10	500000	1	Token hợp lệ User 1	- HTTP Code: 200 OK - Response: JSON áp dụng mã giảm giá thành công. - UI: Hiển thị giá mới đã chiết khấu.	- HTTP Code: 200 OK - Response: {"success":true,"code":200,"message":"Áp dụng thành công! Giảm 50,000 VND."} - UI: Hiển thị thông báo thành công

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	code	total_amount	user_id	jwt_token	Kết quả mong đợi (Expected Output)	Kết quả thực tế (Actual Output)
TC-BVA-07	Áp dụng mã giảm giá vừa hết hạn (hết hạn 1 giây)	<i>expired_at</i> + 1s của mã <i>SAVE10</i>	Mã <i>SAVE10</i> đang hoạt động. Thời gian hệ thống hiện tại là sát sau giờ hết hạn 1 giây.	<i>SAVE10</i>	500000	1	Token hợp lệ User 1	- HTTP Code: 400 Bad Request - Response: JSON báo lỗi mã giảm giá đã hết hạn. - UI: Hiển thị thông báo mã giảm giá hết hạn.	- HTTP Code: 400 Bad Request - Response: {"error": "Invalid coupon code"} - UI: Hiển thị thông báo lỗi mã giảm giá.

Bước 5: Phân tích khoảng trống AI (AI Gap Analysis)

1. Các kịch bản/lỗi kiểm thử mà AI đã bỏ sót

- Race Condition** khi gửi đồng thời nhiều request áp dụng mã (Double Apply): AI khi đọc tài liệu tĩnh thường bỏ qua kịch bản người dùng nhấp đúp nhanh hoặc dùng script gửi đồng thời nhiều yêu cầu áp dụng mã giảm giá (ví dụ gửi 2 request trong cùng 1ms cho mã có giới hạn 1 lần sử dụng). Từ góc nhìn hộp đen, cần kiểm tra liệu chỉ một request được chấp nhận hay nhiều request đều trả về áp dụng thành công.
- Bypass kiểm tra ngưỡng tối thiểu tại bước tạo đơn hàng (Checkout bypass)**: Một lỗi logic nghiệp vụ nghiêm trọng mà AI ít khi nghĩ tới là: Người dùng thêm sản phẩm vào giỏ để tổng tiền đạt 300,000 VND, áp dụng mã *SAVE10* thành công. Sau đó, họ xóa bớt sản phẩm trong giỏ hàng để tổng tiền giảm xuống còn 100,000 VND rồi tiến hành tạo đơn hàng. Từ góc nhìn hộp đen, cần quan sát xem API checkout có từ chối/tự tính lại tổng tiền theo giỏ hiện tại hay vẫn tạo đơn với số tiền client gửi lên.
- Bất đồng bộ múi giờ (Timezone discrepancies)**: AI thường không thiết kế các test case cho sự chênh lệch múi giờ giữa client và server. Ví dụ: coupon hết hạn vào lúc **2026-07-05 23:59:59** theo múi giờ GMT+7, nhưng server chạy GMT+0 hoặc client điều chỉnh giờ hệ thống để cố tình áp dụng mã đã hết hạn.

2. Các sự nhầm lẫn, ảo giác và thiếu sót của AI trong quá trình thiết kế (AI Critique)

- Nhầm lẫn System State thành Direct Input**: AI có xu hướng liệt kê *user_coupon_usage* (số lần đã dùng mã của user) hay *coupon_expiration* (trạng thái hết hạn) vào cột Input của bảng test case, yêu cầu người dùng phải truyền các giá trị này trong request body. Trên thực tế, đây là trạng thái hệ thống cần truy vấn từ DB, do đó bắt buộc phải nằm ở cột Preconditions.
- Lỗi ảo giác về kiểm chứng trực tiếp Database (Grey-box Bias)**: AI đề xuất Expected Output chứa việc kiểm tra dữ liệu trực tiếp trong CSDL (ví dụ: "Kiểm tra bảng *coupon_usages* có thêm dòng mới"). Điều này vi phạm nguyên tắc kiểm thử hộp đen tính khi chỉ được phép quan sát hành vi thông qua HTTP response hoặc UI.
- Xu hướng thiên vị cài đặt cụ thể (Implementation Bias)**: AI định nghĩa chi tiết các chuỗi thông báo lỗi JSON (ví dụ: **{"status": "error", "message": "Min amount not reached"}**) vào Expected Output thay vì mô tả hành vi nghiệp vụ ở mức tổng quát. Điều này khiến bộ test case dễ bị lỗi thời nếu định dạng API thay thế chuỗi thông báo lỗi.
- Lỗi thiết kế giá trị biên trùng lặp gây che khuất lỗi (Test Masking / Boundary Collision)**: Khi thiết kế kịch bản BVA để kiểm thử biên cho một biến (ví dụ: số lần sử dụng *VIP100* của user tại *TC-BVA-04* và *TC-BVA-05*), AI lại chọn giá trị đầu vào phụ (*total_amount* = 300,000 VND) đúng bằng giá trị biên tối thiểu của biến đó thay vì chọn các giá trị thông thường (nominal value) tuyệt đối an toàn (ví dụ: 500,000 VND). Thiết kế này có thể khiến lỗi ở biên tối thiểu của *total_amount* che khuất hành vi của biến cần kiểm thử, làm giảm hiệu quả kiểm định độc lập của từng biến.

3. Giải thích nguyên nhân AI gặp các hạn chế trên

- Thiếu khả năng thực thi và kiểm nghiệm động**: AI chỉ làm việc trên tài liệu đặc tả dạng văn bản và suy luận tĩnh. Do đó, AI không thể tự động nhận thức được các vấn đề phát sinh trong môi trường chạy thực tế như độ trễ mạng, xử lý đa luồng bất đồng bộ (concurrency), hay xung đột tranh chấp ghi dữ liệu.
- Thiếu tư duy tấn công bảo mật (Security Threat Modeling)**: AI thường chỉ tập trung tối ưu hóa các luồng đi bình thường (Happy Path) và các lỗi nhập liệu đơn giản trên form, mà không chủ động đặt giả thuyết về việc người dùng cố tình thay đổi tham số request gửi trực tiếp qua Postman/cURL để qua mặt hệ thống.

Pool C: FR-17: Quản lý Mã Giảm Giá (Coupon CRUD)

Bước 1: Xác định các biến Input và Output (I/O Variables)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các biến vào/ra của tính năng **FR-17: Quản lý Mã Giảm Giá (Coupon CRUD)**, em đã thực hiện các bước phân tích sau:

- Phân tích đặc tả nghiệp vụ (Specification Analysis)**: Đọc đặc tả FR-17 trong [eshop-sut/README.md](#) và đặc tả API trong [eshop-sut/api_specification.md](#). Chức năng cho phép Admin thêm, xem và xóa mã giảm giá; đồng thời chịu yêu cầu kiểm soát truy cập Admin của FR-12.
- Xác định biến đầu vào trực tiếp (Direct Inputs)**: Các dữ liệu người kiểm thử có thể gửi qua UI Admin hoặc API gồm thao tác CRUD, token xác thực, *code*, *type*, *discount_value*, *expired_at*, *min_order_amount*, *max_uses_per_user* và *coupon_id*.
- Xác định biến đầu vào trạng thái (System State Inputs)**: Vai trò Admin trong token và trạng thái tồn tại/duy nhất của coupon là trạng thái hệ thống, không phải trường nhập form. Vì vậy các biến này được đưa vào **Điều kiện tiên đề (Preconditions)** khi thiết kế test case.
- Xác định biến đầu ra (Outputs)**: Hệ thống phản hồi bằng HTTP status code, nội dung JSON ở mức API, thông báo UI và hành động UI tương ứng như cập nhật danh sách hoặc hiển thị lỗi.

1. Các biến đầu vào (Input Variables)

Dưới đây là danh sách biến đầu vào của chức năng quản lý mã giảm giá:

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	operation	Direct Input	Enum	view, create, delete	Thao tác quản trị mã giảm giá cần thực hiện.
2	authorization_token	Direct Input	String	Header Authorization: Bearer <token> hợp lệ	Token JWT dùng để gọi API Admin hoặc API có tác động dữ liệu.
3	code	Direct Input	String	Bắt buộc, duy nhất	Mã giảm giá khi tạo mới coupon.
4	type	Direct Input	Enum	percent hoặc fixed	Loại giảm giá theo phần trăm hoặc số tiền cố định.
5	discount_value	Direct Input	Number	Số dương (> 0)	Giá trị giảm giá tương ứng với type.
6	expired_at	Direct Input	Date/String	Bắt buộc, định dạng ngày hợp lệ	Ngày hết hạn của mã giảm giá.
7	min_order_amount	Direct Input	Number	>= 0	Giá trị đơn hàng tối thiểu để sử dụng mã.
8	max_uses_per_user	Direct Input	Integer	>= 1	Số lần tối đa mỗi người dùng được sử dụng mã.
9	coupon_id	Direct Input	Integer	ID coupon tồn tại khi xóa	ID mã giảm giá cần xóa trong endpoint DELETE /api/admin/coupons/:id .
10	admin_role	State Input	Enum	Token phải có role = 'admin'	Quyền Admin được mã hóa trong token — Dùng làm Precondition .
11	coupon_state	State Input	Enum	Coupon chưa tồn tại / đã tồn tại / ID tồn tại	Trạng thái duy nhất của code và tồn tại của coupon_id — Dùng làm Precondition .

2. Các biến đầu ra (Output Variables)

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	http_status_code	API Output	Integer	Thành công / lỗi xác thực / lỗi phân quyền / lỗi validation / không tìm thấy / trùng dữ liệu	Mã phản hồi HTTP cho từng thao tác CRUD.
2	json_response	API Output	JSON Object/Array	Danh sách coupon, coupon mới tạo, kết quả xóa hoặc thông báo lỗi	Nội dung phản hồi API theo kết quả nghiệp vụ.
3	ui_message	UI Output	String	Thông báo thành công hoặc lỗi	Nội dung hiển thị cho Admin trên giao diện.
4	ui_action	UI Output	Enum	Cập nhật danh sách / giữ nguyên form / chặn thao tác	Phản ứng của UI sau khi nhận phản hồi.

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để thực hiện kỹ thuật Phân hoạch tương đương cho tính năng **FR-17: Quản lý Mã Giảm Giá (Coupon CRUD)**, em đã áp dụng các bước có hệ thống sau:

1. **Phân tích điều kiện đầu vào/đầu ra:** Tách riêng các điều kiện của thao tác xem danh sách, thêm mới và xóa coupon, đồng thời đưa yêu cầu quyền Admin của FR-12 vào điều kiện tiền đề.
2. **Xác định các lớp tương đương Valid và Invalid:** Mỗi ràng buộc bắt buộc như **code** duy nhất, **type** thuộc tập hợp, số dương và quyền Admin được chia thành lớp hợp lệ và các lớp vi phạm đặc trưng.
3. **Lựa chọn giá trị đại diện (Representatives):** Chọn giá trị cụ thể như **TET2025**, **percent**, **fixed**, **discount_value = 15**, **min_order_amount = 200000** và các giá trị lỗi như chuỗi rỗng, dữ liệu trùng hoặc sai kiểu.
4. **Thiết kế tập Test Cases tối thiểu:** Một test case valid bao phủ toàn bộ luồng CRUD hợp lệ; các test case invalid còn lại mỗi case chỉ chứa một lớp invalid để tránh che giấu lỗi.

1. Các biến đầu vào (Input Variables)

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC01	operation	Valid	Thao tác thuộc nhóm view, create, delete	Admin thực hiện đúng thao tác CRUD được đặc tả.

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC02	authorization_token	Valid	Token JWT hợp lệ, chưa hết hạn	Request có thông tin xác thực hợp lệ.
EC03	admin_role	Valid	Token có role = 'admin'	Người gọi có quyền Admin theo FR-12.
EC04	code	Valid	code không rỗng và chưa tồn tại	Có thể tạo mã giảm giá mới.
EC05	type	Valid	type là percent hoặc fixed	Loại coupon nằm trong tập cho phép.
EC06	discount_value	Valid	Giá trị số dương (> 0)	Đáp ứng ràng buộc giá trị giảm phải dương.
EC07	expired_at	Valid	Có ngày hết hạn và định dạng ngày hợp lệ	Coupon có thông tin hết hạn hợp lệ.
EC08	min_order_amount	Valid	Giá trị số >= 0	Đáp ứng ràng buộc giá trị đơn tối thiểu không âm.
EC09	max_uses_per_user	Valid	Số nguyên >= 1	Có ít nhất một lượt sử dụng cho mỗi user.
EC10	coupon_id	Valid	ID coupon tồn tại trong hệ thống	Có thể xóa đúng coupon đang tồn tại.
EC11	operation	Invalid	Thao tác không thuộc CRUD được hỗ trợ	Chặn thao tác ngoài đặc tả.
EC12	authorization_token	Invalid	Thiếu token xác thực	Chặn request chưa đăng nhập.
EC13	authorization_token	Invalid	Token sai định dạng, giả mạo hoặc hết hạn	Chặn request có token không hợp lệ.
EC14	admin_role	Invalid	Token hợp lệ nhưng không có role Admin	Chặn user thường truy cập chức năng Admin.
EC15	code	Invalid	code rỗng hoặc không truyền	Trường bắt buộc bị thiếu.
EC16	code	Invalid	code đã tồn tại	Vi phạm yêu cầu duy nhất.
EC17	type	Invalid	type ngoài tập percent/fixed	Từ chối loại coupon không được hỗ trợ.
EC18	type	Invalid	Thiếu trường type	Trường bắt buộc bị thiếu.
EC19	discount_value	Invalid	Bằng 0	Vi phạm điều kiện số dương.
EC20	discount_value	Invalid	Nhỏ hơn 0	Vi phạm điều kiện số dương.
EC21	discount_value	Invalid	Thiếu hoặc không phải số	Dữ liệu sai kiểu hoặc thiếu trường bắt buộc.
EC22	expired_at	Invalid	Thiếu ngày hết hạn	Trường bắt buộc bị thiếu.
EC23	expired_at	Invalid	Định dạng ngày không hợp lệ	Không thể diễn giải ngày hết hạn.
EC24	min_order_amount	Invalid	Nhỏ hơn 0	Vi phạm ràng buộc không âm.
EC25	min_order_amount	Invalid	Thiếu hoặc không phải số	Dữ liệu sai kiểu hoặc thiếu trường bắt buộc.
EC26	max_uses_per_user	Invalid	Thiếu, không phải số nguyên hoặc nhỏ hơn 1	Vi phạm ràng buộc số lần dùng tối thiểu.
EC27	coupon_id	Invalid	ID không tồn tại, thiếu hoặc không hợp lệ	Xóa coupon không xác định hoặc ngoài miền hợp lệ.

2. Các biến đầu ra (Output Variables)

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC28	http_status_code	Valid (Success)	Thành công khi xem, tạo hoặc xóa coupon	Hệ thống xử lý thao tác hợp lệ.
EC29	http_status_code	Invalid (Failure)	Lỗi validation dữ liệu đầu vào	Trường bắt buộc, sai kiểu hoặc ngoài miền.
EC30	http_status_code	Invalid (Failure)	Lỗi chưa xác thực	Thiếu hoặc sai token.
EC31	http_status_code	Invalid (Failure)	Lỗi không đủ quyền	User không phải Admin.
EC32	http_status_code	Invalid (Failure)	Không tìm thấy coupon cần xóa	ID không tồn tại.
EC33	http_status_code	Invalid (Failure)	Xung đột dữ liệu duy nhất	code bị trùng.
EC34	json_response	Valid (Success)	JSON chứa danh sách coupon hoặc coupon vừa tạo	Dữ liệu phản hồi thành công.
EC35	json_response	Valid (Success)	JSON xác nhận thao tác xóa thành công	Phản hồi sau mutation hợp lệ.
EC36	json_response	Invalid (Failure)	JSON thông báo lỗi nghiệp vụ hoặc bảo mật	Phản hồi khi request bị từ chối.

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC37	ui_message & ui_action	Valid/Invalid	UI cập nhật danh sách khi thành công hoặc hiển thị lỗi và giữ nguyên dữ liệu khi thất bại	Hành vi giao diện tương ứng kết quả API.

Bước 3: Lựa chọn giá trị đại diện (Selecting Representatives)

1. Bảng giá trị đại diện cho các lớp tương đương

Mã lớp	Biến tương ứng	Loại lớp	Giá trị đại diện	Ý nghĩa / Ghi chú kiểm thử
EC01	operation	Valid	view, create, delete	Ba thao tác CRUD được đặc tả cho Admin.
EC02	authorization_token	Valid	Token hợp lệ của Admin	Request đã xác thực.
EC03	admin_role	Valid	role = 'admin'	Có quyền quản trị theo FR-12.
EC04	code	Valid	TET2025	Mã mới, chưa tồn tại.
EC05	type	Valid	percent / fixed	Hai loại coupon hợp lệ.
EC06	discount_value	Valid	15	Giá trị giảm dương.
EC07	expired_at	Valid	2027-01-31	Ngày hợp lệ.
EC08	min_order_amount	Valid	200000	Giá trị không âm.
EC09	max_uses_per_user	Valid	1	Số nguyên tối thiểu hợp lệ.
EC10	coupon_id	Valid	1	Coupon tồn tại để xóa.
EC11	operation	Invalid	update	Thao tác không được FR-17 mô tả.
EC12	authorization_token	Invalid	Không truyền header Authorization	Thiếu xác thực.
EC13	authorization_token	Invalid	Token giả hoặc hết hạn	Xác thực không hợp lệ.
EC14	admin_role	Invalid	Token của user thường	Không đủ quyền Admin.
EC15	code	Invalid	""	Bỏ trống mã.
EC16	code	Invalid	SAVE10 đã tồn tại	Vi phạm duy nhất.
EC17	type	Invalid	cashback	Loại coupon ngoài tập cho phép.
EC18	type	Invalid	Không truyền type	Thiếu trường bắt buộc.
EC19	discount_value	Invalid	0	Không phải số dương.
EC20	discount_value	Invalid	-5	Giá trị âm.
EC21	discount_value	Invalid	"abc" hoặc thiếu trường	Sai kiểu dữ liệu hoặc thiếu trường.
EC22	expired_at	Invalid	Không truyền expired_at	Thiếu trường bắt buộc.
EC23	expired_at	Invalid	not-a-date	Sai định dạng ngày.
EC24	min_order_amount	Invalid	-1	Nhỏ hơn 0.
EC25	min_order_amount	Invalid	"two hundred" hoặc thiếu trường	Sai kiểu dữ liệu hoặc thiếu trường.
EC26	max_uses_per_user	Invalid	0, 1.5, hoặc thiếu trường	Không đạt miền số nguyên >= 1.
EC27	coupon_id	Invalid	0 hoặc 999999	ID không hợp lệ hoặc không tồn tại.
EC28	http_status_code	Success	Thành công	API xử lý thao tác hợp lệ.
EC29	http_status_code	Failure	Lỗi validation	Dữ liệu đầu vào bị từ chối.
EC30	http_status_code	Failure	Lỗi xác thực	Thiếu/sai token.
EC31	http_status_code	Failure	Lỗi phân quyền	Không phải Admin.
EC32	http_status_code	Failure	Không tìm thấy	Xóa coupon không tồn tại.
EC33	http_status_code	Failure	Xung đột dữ liệu	Trùng code.
EC34	json_response	Success	Danh sách coupon hoặc coupon mới	Dữ liệu trả về khi xem/tạo thành công.
EC35	json_response	Success	Xác nhận xóa thành công	Dữ liệu trả về sau khi xóa hợp lệ.
EC36	json_response	Failure	Thông báo lỗi nghiệp vụ/bảo mật	Phản hồi lỗi tổng quát theo loại lỗi.
EC37	ui_message & ui_action	Valid/Invalid	Cập nhật danh sách hoặc hiển thị lỗi	Phản ứng UI tương ứng API.

2. Thiết kế tập Test Cases phân hoạch tương đương (Equivalence Partitioning Test Cases)

Tập test cases tối thiểu dưới đây được thiết kế nhằm bao phủ toàn bộ các lớp tương đương đã phân hoạch ở Bước 2:

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiền đề (Preconditions)	operation	authorization_token	code	type	discount
TC01	Admin thực hiện đầy đủ luồng xem, thêm và xóa coupon hợp lệ	EC01, EC02, EC03, EC04, EC05, EC06, EC07, EC08, EC09, EC10, EC28, EC34, EC35, EC37	Token thuộc Admin. TET2025 và FREESHIP50 chưa tồn tại; coupon_id = 1 tồn tại để xóa.	view →	Token Admin hợp lệ	TET2025, FREESHIP50	percent, fixed	15, 50%
				create → create → delete				
TC02	Từ chối thao tác không thuộc CRUD được hỗ trợ	EC11, EC29, EC36, EC37	Token thuộc Admin. Dữ liệu coupon còn lại hợp lệ.	update	Token Admin hợp lệ	TET2025	percent	15
TC03	Từ chối request không có token	EC12, EC30, EC36, EC37	Không có phiên đăng nhập Admin. Dữ liệu tạo coupon hợp lệ.	create	Không truyền	TET2025	percent	15

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discou
TC04	Từ chối token sai hoặc hết hạn	EC13, EC30, EC36, EC37	Token bị sửa đổi, hết hạn hoặc không thể xác minh. Dữ liệu tạo coupon hợp lệ.	create	Token không hợp lệ	TET2025	percent	15
TC05	Từ chối user thường truy cập chức năng Admin	EC14, EC31, EC36, EC37	Token hợp lệ nhưng thuộc user không có role Admin. Dữ liệu tạo coupon hợp lệ.	create	Token user thường	TET2025	percent	15
TC06	Từ chối tạo coupon thiếu code	EC15, EC29, EC36, EC37	Token thuộc Admin. type, discount_value, expired_at, min_order_amount, max_uses_per_user hợp lệ.	create	Token Admin hợp lệ	""	percent	15
TC07	Từ chối tạo coupon có code trùng	EC16, EC33, EC36, EC37	Token thuộc Admin. Mã SAVE10 đã tồn tại trong hệ thống. Các trường khác hợp lệ.	create	Token Admin hợp lệ	SAVE10	percent	15

Mã TC	Tên Test Case	Lớp tương đương phù	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discount
TC08	Từ chối type ngoài tập cho phép	EC17, EC29, EC36, EC37	Token thuộc Admin.					
			code chưa tồn tại, các trường số và ngày hợp lệ.	create	Token Admin hợp lệ	TET2025	cashback	15
TC09	Từ chối tạo coupon thiếu type	EC18, EC29, EC36, EC37	Token thuộc Admin.					
			Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	Không truyền	15
TC10	Từ chối discount_value bằng 0	EC19, EC29, EC36, EC37	Token thuộc Admin.					
			code chưa tồn tại, type , ngày hết hạn và các giới hạn khác hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	0
TC11	Từ chối discount_value âm	EC20, EC29, EC36, EC37	Token thuộc Admin.					
			Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	fixed	-5

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiền đề (Preconditions)	operation	authorization_token	code	type	discou
TC12	Từ chối <code>discount_value</code> thiếu hoặc sai kiểu	EC21, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	"abc"
TC13	Từ chối thiếu <code>expired_at</code>	EC22, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15
TC14	Từ chối <code>expired_at</code> sai định dạng	EC23, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15
TC15	Từ chối <code>min_order_amount</code> âm	EC24, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discou
TC16	Từ chối min_order_amount	EC25, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	fixed	50000
	thiếu hoặc sai kiểu							
TC17	Từ chối max_uses_per_user	EC26, EC29, EC36, EC37	Token thuộc Admin. Các trường còn lại hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15
	không đạt miễn hợp lệ							
TC18	Từ chối xóa coupon không tồn tại hoặc ID không hợp lệ	EC27, EC32, EC36, EC37	Token thuộc Admin. Không có coupon với ID được gửi hoặc ID không thuộc miễn hợp lệ.	delete	Token Admin hợp lệ	N/A	N/A	N/A

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiền đề (Preconditions)	operation	authorization_token	code	type	discou
TC19	Kiểm thử đồng thời tạo hai coupon trùng code	EC16, EC33, EC36, EC37 (integration/concurrency)	Token thuộc Admin. RACE2027 chưa tồn tại trước khi bắt đầu; gửi đồng thời 2 request tạo cùng mã.	create	Token Admin hợp lệ	RACE2027	percent	15
				song song				

Bước 4: Phân tích giá trị biên (Boundary Value Analysis - BVA)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các giá trị biên nhạy cảm của tính năng **FR-17: Quản lý Mã Giảm Giá (Coupon CRUD)**, em đã thực hiện phân tích theo các bước sau:

1. **Xác định các biến có tính thứ tự hoặc khoảng số:** Các biến phù hợp BVA gồm độ dài `code`, `discount_value`, `min_order_amount`, `max_uses_per_user` và `coupon_id`.
2. **Xác định các điểm biên (Boundaries) cho từng biến:** Dựa trên ràng buộc bắt buộc, duy nhất, số dương, ≥ 0 , ≥ 1 , và ID hợp lệ khi xóa.
3. **Lựa chọn các điểm kiểm thử biên nhạy cảm:** Chọn các điểm $0/1$, $-1/0$, và ID $0/1$ theo đơn vị nguyên nhỏ nhất có thể gửi qua API/UI công khai.

1. Phân tích giá trị biên của các biến số/khoảng số

- **Biến `code_length` (Khoảng hợp lệ tối thiểu: $[1, +\infty)$ ký tự):**

◦ $LB = 1$: Mã có đúng 1 ký tự, tối thiểu hợp lệ.

◦ $LB - 1 = 0$: Chuỗi rỗng, không hợp lệ.
- **Biến `discount_value` (Khoảng hợp lệ: $(0, +\infty)$):**

◦ $LB = 1$: Giá trị dương nhỏ nhất theo đơn vị nguyên, hợp lệ.

◦ $LB - 1 = 0$: Không phải số dương, không hợp lệ.
- **Biến `min_order_amount` (Khoảng hợp lệ: $[0, +\infty)$):**

◦ $LB = 0$: Không yêu cầu giá trị đơn tối thiểu, hợp lệ.

◦ $LB - 1 = -1$: Giá trị âm, không hợp lệ.
- **Biến `max_uses_per_user` (Khoảng hợp lệ: $[1, +\infty)$):**

◦ $LB = 1$: Cho phép mỗi user dùng 1 lần, hợp lệ.

◦ $LB - 1 = 0$: Không cho phép lượt sử dụng nào, không hợp lệ.
- **Biến `coupon_id` khi xóa (Khoảng ID hợp lệ công khai: $[1, +\infty)$ với điều kiện ID tồn tại):**

◦ $LB = 1$: ID dương và tồn tại, hợp lệ.

◦ $LB - 1 = 0$: ID không hợp lệ để xóa coupon.

2. Thiết kế tập Test Cases giá trị biên (Boundary Value Test Cases)

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discount_value	expected
TC-BVA-01	Tạo coupon với code rỗng	code_length = LB-1 = 0	Token thuộc Admin. Các trường khác hợp lệ.	create	Token Admin hợp lệ	""	percent	15	263%
TC-BVA-02	Tạo coupon với code dài 1 ký tự	code_length = LB = 1	Token thuộc Admin. Mã A chưa tồn tại. Các trường khác hợp lệ.	create	Token Admin hợp lệ	A	percent	15	263%
TC-BVA-03	Tạo coupon với discount_value = 0	discount_value = LB-1 = 0	Token thuộc Admin. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	0	263%
TC-BVA-04	Tạo coupon với discount_value = 1	discount_value = LB = 1	Token thuộc Admin. TET2025 chưa tồn tại. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	1	263%

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discount_value	expected
TC-BVA-05	Tạo coupon với min_order_amount = -1	min_order_amount = LB-1 = -1	Token thuộc Admin. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	fixed	50000	20000
TC-BVA-06	Tạo coupon với min_order_amount = 0	min_order_amount = LB = 0	Token thuộc Admin. TET2025 chưa tồn tại. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	fixed	50000	20000
TC-BVA-07	Tạo coupon với max_uses_per_user = 0	max_uses_per_user = LB-1 = 0	Token thuộc Admin. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15	20
TC-BVA-08	Tạo coupon với max_uses_per_user = 1	max_uses_per_user = LB = 1	Token thuộc Admin. TET2025 chưa tồn tại. Các trường khác hợp lệ.	create	Token Admin hợp lệ	TET2025	percent	15	20

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	operation	authorization_token	code	type	discount_value	expected
TC-BVA-09	Xóa coupon với <code>coupon_id = 0</code>	<code>coupon_id = LB-1 = 0</code>	Token thuộc Admin. Không có coupon hợp lệ với ID <code>0</code> .	<code>delete</code>	Token Admin hợp lệ	N/A	N/A	N/A	N/A
TC-BVA-10	Xóa coupon với <code>coupon_id = 1</code> tồn tại	<code>coupon_id = LB = 1</code>	Token thuộc Admin. Coupon có ID <code>1</code> tồn tại và được phép xóa.	<code>delete</code>	Token Admin hợp lệ	N/A	N/A	N/A	N/A

Bước 5: Phân tích khoảng trống AI (AI Gap Analysis)

1. Các kịch bản/lỗi kiểm thử mà AI đã bỏ sót

- **Race Condition khi tạo trùng mã coupon:** AI thường chỉ kiểm thử trùng `code` tuần tự, nhưng bỏ sót tình huống 2 Admin hoặc 2 request API tạo cùng một mã trong cùng thời điểm. Nếu hệ thống không kiểm soát unique ở mức giao dịch, có thể xuất hiện hai coupon cùng `code`.
- **Bypass quyền Admin qua API trực tiếp:** Giao diện Admin có thể ẩn màn hình với user thường, nhưng request trực tiếp tới `POST /api/admin/coupons` hoặc `DELETE /api/admin/coupons/:id` vẫn cần bị chặn bởi token và role Admin theo FR-12.
- **Xóa coupon đang được tham chiếu trong luồng sử dụng khác:** Đặc tả chỉ nêu Admin có thể xóa mã, nhưng AI có thể bỏ qua rủi ro xóa mã đang được người dùng nhìn thấy hoặc vừa áp dụng trong phiên checkout, dẫn đến sai lệch trạng thái giữa Admin và khách hàng.

2. Các sự nhầm lẫn, ảo giác và thiếu sót của AI trong quá trình thiết kế (AI Critique)

- **Nhầm System State thành Direct Input:** AI dễ đưa `admin_role` hoặc trạng thái `code` đã tồn tại thành cột input gửi trong body. Thực tế đây là trạng thái hệ thống và phải nằm ở Preconditions.
- **Implementation Bias về status code và chuỗi lỗi:** AI có xu hướng ghi cứng từng chuỗi JSON hoặc status code cụ thể. Với kiểm thử hộp đen từ đặc tả, Expected Output nên mô tả hành vi nghiệp vụ như “bị từ chối do thiếu quyền” hoặc “bị từ chối do trùng mã”.
- **Bỏ sót endpoint xem danh sách không nằm dưới /api/admin/*:** API lấy danh sách coupon dùng `GET /api/coupons` nhưng vẫn ghi “Dành cho Admin” và cần header Authorization. AI có thể chỉ tập trung vào `/api/admin/coupons` mà không kiểm thử quyền truy cập danh sách.

3. Giải thích nguyên nhân AI gặp các hạn chế trên

- **Đặc tả ngắn và có nhiều ràng buộc ngầm:** FR-17 chỉ mô tả CRUD và các trường bắt buộc ở mức tóm tắt, nên AI phải suy luận cẩn thận từ FR-12 và API spec để không bỏ sót quyền Admin.
- **Khó phân biệt validation nghiệp vụ với trạng thái hệ thống:** Các khái niệm như `code` duy nhất hoặc `coupon_id` tồn tại không phải giá trị form thuần túy mà phụ thuộc dữ liệu hệ thống, khiến AI dễ mô hình hóa sai.

- **Thiếu quan sát động trong giai đoạn thiết kế:** Ở thời điểm thiết kế, AI chưa thể biết thông điệp lỗi và hành vi UI thật; sau khi thực thi black-box, Actual Output và Pass/Fail được bổ sung dựa trên quan sát UI/API công khai.

Pool D: FR-07: Giỏ hàng (Shopping Cart)

Bước 1: Xác định các biến Input và Output (I/O Variables)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các biến vào/ra của tính năng **FR-07: Giỏ hàng (Shopping Cart)**, em đã thực hiện các bước phân tích sau:

- Phân tích đặc tả nghiệp vụ (Specification Analysis):** Đọc phần FR-07 trong [README.md](#), tính năng giỏ hàng yêu cầu hiển thị danh sách sản phẩm, đơn giá, số lượng có nút **+/-**, thành tiền, thao tác xóa có xác nhận, nút tiếp tục mua sắm, nhân tổng tiền chính xác và trạng thái giỏ hàng trống rõ ràng. Đọc thêm [api_specification.md](#), API giỏ hàng gồm **GET /api/cart** và **POST /api/cart**, yêu cầu header Authorization.
- Xác định biến đầu vào trực tiếp (Direct Inputs):** Các giá trị người dùng hoặc client gửi trực tiếp gồm **authorization_token**, **cart_action**, **product_id**, **product_name**, **unit_price**, **quantity** và **delete_confirmation**.
- Xác định biến đầu vào trạng thái (System State Inputs):** Một số điều kiện không phải dữ liệu nhập trực tiếp nhưng quyết định nhánh xử lý gồm **cart_state**, **product_already_in_cart** và **current_quantity**. Các biến này được đặt trong cột Preconditions khi thiết kế test case.
- Xác định biến đầu ra (Outputs):** Hệ thống cần phản hồi ở cả API và UI: HTTP status/JSON, bảng giỏ hàng, số lượng, thành tiền từng dòng, tổng tiền với nhãn **"Tổng cộng"**, dialog xác nhận xóa, điều hướng quay về trang chủ và trạng thái giỏ hàng trống có hình minh họa/thông báo rõ ràng.

1. Các biến đầu vào (Input Variables)

Bao gồm các biến người dùng/client nhập trực tiếp và các biến trạng thái hệ thống ảnh hưởng đến hành vi giỏ hàng:

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	authorization_token	Direct Input	String	Header Authorization: Bearer <token> bắt buộc cho API giỏ hàng.	Token xác thực người dùng khi gọi API giỏ hàng.
2	cart_action	Direct Input	Enum	Các hành động hợp lệ: xem giỏ, thêm sản phẩm, tăng/giảm số lượng, xóa sản phẩm, tiếp tục mua sắm.	Thao tác người dùng thực hiện trên giỏ hàng.
3	product_id	Direct Input	Integer/String	ID sản phẩm phải xác định được sản phẩm cần thêm/cập nhật/xóa.	Định danh sản phẩm trong giỏ hàng.
4	product_name	Direct Input	String	Tên sản phẩm phải hiển thị trong cột Sản phẩm .	Tên sản phẩm được hiển thị trong giỏ.
5	unit_price	Direct Input	Number	Đơn giá phải là số tiền hợp lệ để tính thành tiền và tổng cộng.	Giá của một đơn vị sản phẩm.
6	quantity	Direct Input	Integer	Số lượng là số nguyên dương, tối thiểu 1 ; UI có nút +/- để chỉnh.	Số lượng sản phẩm trong giỏ hàng.
7	delete_confirmation	Direct Input	Boolean/Enum	Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện.	Quyết định xác nhận hoặc hủy thao tác xóa.
8	cart_state	State Input	Enum	Giỏ hàng có thể rỗng hoặc có ít nhất một sản phẩm.	Trạng thái dữ liệu giỏ hàng trước thao tác.
9	product_already_in_cart	State Input	Boolean	Nếu cùng một sản phẩm đã có trong giỏ, thêm tiếp phải tăng số lượng, không tạo dòng mới.	Trạng thái tồn tại của sản phẩm trong giỏ.
10	current_quantity	State Input	Integer	Số lượng hiện tại tối thiểu là 1 ; thao tác giảm không được tạo số lượng nhỏ hơn 1 .	Số lượng hiện tại trước khi bấm nút +/- .

2. Các biến đầu ra (Output Variables)

Bao gồm phản hồi API và các thay đổi quan sát được trên giao diện giỏ hàng:

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
1	http_status_code	API Output	Integer	Thành công khi thao tác hợp lệ; lỗi khi thiếu xác thực hoặc dữ liệu không hợp lệ.	Mã trạng thái HTTP từ API giỏ hàng.
2	api_response_payload	API Output	JSON Object	Thành công trả dữ liệu giỏ hàng/cập nhật; thất bại trả thông báo lỗi nghiệp vụ.	Nội dung phản hồi API.
3	cart_table	UI Output	Table	Khi có sản phẩm, hiển thị đủ cột Sản phẩm, Đơn giá, Số lượng, Thành tiền, Thao tác .	Bảng danh sách sản phẩm trong giỏ.
4	quantity_display	UI Output	Integer	Hiển thị số lượng hiện tại sau thao tác thêm/tăng/giảm.	Số lượng quan sát trên giao diện.

STT	Tên biến	Loại biến	Kiểu dữ liệu	Ràng buộc đặc tả / Miền giá trị	Mô tả
5	line_total	UI Output	Number	Thành tiền từng dòng = đơn giá x số lượng.	Giá trị thành tiền theo từng sản phẩm.
6	total_label	UI Output	String	Nhấn tổng tiền phải là " Tổng cộng ", không phải " Tổng tạm tính ".	Nhấn tổng tiền của giỏ hàng.
7	delete_confirmation_dialog	UI Output	Dialog	Phải xuất hiện trước khi xóa sản phẩm.	Hộp thoại xác nhận thao tác xóa.
8	navigation_action	UI Output	Enum	Nút Tiếp tục mua sắm quay về trang chủ.	Hành động điều hướng sau khi bấm nút.
9	empty_cart_state	UI Output	UI State	Giỏ hàng trống phải có hình minh họa và thông báo rõ ràng.	Trạng thái giao diện khi không có sản phẩm.

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để thực hiện kỹ thuật Phân hoạch tương đương cho tính năng **FR-07: Giỏ hàng (Shopping Cart)**, em đã áp dụng các bước sau:

- Phân tích điều kiện đầu vào/đầu ra:** Tách các điều kiện theo thao tác giỏ hàng: xác thực API, thêm sản phẩm, cộng dồn sản phẩm đã có, chỉnh số lượng, xóa có xác nhận, tiếp tục mua sắm và hiển thị giỏ trống.
- Xác định các lớp tương đương Valid và Invalid:** Với **quantity**, phân tách rõ số nguyên dương hợp lệ, rỗng, sai kiểu, bằng **0** và âm. Với trạng thái hệ thống, phân tách giỏ rỗng/không rỗng, sản phẩm đã có/chưa có, và biên giảm số lượng tại **1**.
- Lựa chọn giá trị đại diện (Representatives):** Chọn các giá trị để quan sát như sản phẩm ID **1**, giá **100000**, số lượng **1, 2, 0, -1** để tính toán thành tiền và tổng cộng.
- Thiết kế tập Test Cases tối thiểu:** Thiết kế một test case luồng chính, sau đó tách riêng từng lớp lỗi hoặc nhánh nghiệp vụ quan trọng để tránh che giấu lỗi, đồng thời bổ sung các kịch bản UI đặc thù của FR-07.

1. Các biến đầu vào (Input Variables)

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC01	authorization_token	Valid	Token người dùng hợp lệ	Cho phép truy cập API giỏ hàng.
EC02	authorization_token	Invalid	Token thiếu, hết hạn hoặc sai định dạng	API giỏ hàng phải từ chối truy cập không xác thực.
EC03	cart_action	Valid	Hành động thuộc tập hỗ trợ: view/add/increase/decrease/delete/continue	Người dùng thực hiện thao tác giỏ hàng hợp lệ.
EC04	cart_action	Invalid	Hành động ngoài tập hỗ trợ hoặc endpoint/phương thức không phù hợp	Hệ thống không được xử lý thao tác không xác định.
EC05	product_id	Valid	ID sản phẩm tồn tại/có thể xác định	Sản phẩm được thêm hoặc thao tác đúng dòng.
EC06	product_id	Invalid	ID thiếu, sai định dạng hoặc không xác định được sản phẩm	Hệ thống phải từ chối hoặc không thay đổi giỏ hàng.
EC07	product_name	Valid	Chuỗi tên sản phẩm khác rỗng	Cột Sản phẩm hiển thị rõ tên.
EC08	product_name	Invalid	Tên sản phẩm rỗng hoặc thiếu	Không đủ dữ liệu hiển thị dòng giỏ hàng hợp lệ.
EC09	unit_price	Valid	Đơn giá là số tiền dương	Có thể tính thành tiền và tổng cộng.
EC10	unit_price	Invalid	Đơn giá bằng 0, âm hoặc sai kiểu	Không được chấp nhận giá trị tiền không hợp lệ.
EC11	quantity	Valid	Số nguyên dương >= 1	Số lượng hợp lệ theo đặc tả.
EC12	quantity	Invalid	Rỗng hoặc thiếu	Không đủ dữ liệu số lượng.
EC13	quantity	Invalid	Sai kiểu, không phải số nguyên	V phạm kiểu dữ liệu của số lượng.
EC14	quantity	Invalid	Bằng 0	V phạm ràng buộc tối thiểu là 1.
EC15	quantity	Invalid	Nhỏ hơn 0	V phạm miền số nguyên dương.
EC16	delete_confirmation	Valid	Người dùng xác nhận xóa	Cho phép xóa sản phẩm khỏi giỏ.
EC17	delete_confirmation	Valid	Người dùng hủy xóa	Giỏ hàng phải giữ nguyên.
EC18	cart_state	Valid	Giỏ hàng có ít nhất một sản phẩm	Hiển thị bảng giỏ hàng đầy đủ.
EC19	cart_state	Valid	Giỏ hàng trống	Hiển thị empty state đúng đặc tả.

Mã lớp	Biến đầu vào	Phân loại lớp	Lớp tương đương	Mô tả / Ý nghĩa kiểm thử
EC20	product_already_in_cart	Valid	Sản phẩm chưa có trong giỏ	Thêm sản phẩm tạo một dòng mới.
EC21	product_already_in_cart	Valid	Sản phẩm đã có trong giỏ	Thêm cùng sản phẩm chỉ tăng số lượng, không tạo dòng mới.
EC22	current_quantity	Valid	Số lượng hiện tại lớn hơn 1	Bấm giảm vẫn còn số lượng hợp lệ.
EC23	current_quantity	Invalid	Số lượng hiện tại bằng 1 nhưng vẫn cố giảm tiếp	Không được làm số lượng nhỏ hơn 1.

2. Các biến đầu ra (Output Variables)

Mã lớp	Biến đầu ra	Phân loại lớp	Lớp tương đương	Ý nghĩa phản hồi
EC24	http_status_code	Valid (Success)	HTTP thành công cho thao tác hợp lệ	API xử lý thao tác giỏ hàng thành công.
EC25	http_status_code	Invalid (Failure)	HTTP lỗi xác thực hoặc validation	API từ chối thao tác không hợp lệ.
EC26	api_response_payload	Valid (Success)	JSON thể hiện giỏ hàng đã được lấy/cập nhật	Client có dữ liệu để render giỏ hàng.
EC27	api_response_payload	Invalid (Failure)	JSON thông báo lỗi nghiệp vụ/xác thực	Client nhận phản hồi lỗi phù hợp.
EC28	cart_table	Valid (Success)	Bảng có đủ cột theo đặc tả	UI giỏ hàng đầy đủ thông tin.
EC29	empty_cart_state	Valid (Success)	Có hình minh họa và thông báo giỏ hàng trống	Người dùng hiểu rõ giỏ hiện không có sản phẩm.
EC30	quantity_display / line_total	Valid (Success)	Số lượng và thành tiền cập nhật đúng	Giỏ hàng phản ánh đúng phép tính.
EC31	delete_confirmation_dialog	Valid (Success)	Dialog xác nhận xuất hiện trước khi xóa	Ngăn xóa nhầm sản phẩm.
EC32	navigation_action	Valid (Success)	Điều hướng về trang chủ khi tiếp tục mua sắm	Luồng quay lại mua hàng hoạt động đúng.
EC33	total_label	Valid (Success)	Nhãn tổng tiền hiển thị chính xác "Tổng cộng"	Đáp ứng yêu cầu từ đặc tả, tránh nhãn sai.

Bước 3: Lựa chọn giá trị đại diện (Selecting Representatives)

1. Bảng giá trị đại diện cho các lớp tương đương

Mã lớp	Biến tương ứng	Loại lớp	Giá trị đại diện	Ý nghĩa / Ghi chú kiểm thử
EC01	authorization_token	Valid	Token user hợp lệ	Người dùng đã đăng nhập.
EC02	authorization_token	Invalid	Không truyền token	Kiểm tra yêu cầu xác thực API giỏ hàng.
EC03	cart_action	Valid	add_item	Thao tác giỏ hàng hợp lệ.
EC04	cart_action	Invalid	unsupported_action	Hành động ngoài tập hỗ trợ.
EC05	product_id	Valid	1	Sản phẩm hợp lệ để thêm vào giỏ.
EC06	product_id	Invalid	Không truyền id	Thiếu định danh sản phẩm.
EC07	product_name	Valid	Sản phẩm A	Tên hiển thị hợp lệ.
EC08	product_name	Invalid	""	Tên sản phẩm rỗng.
EC09	unit_price	Valid	100000	Đơn giá dương, dễ tính toán.
EC10	unit_price	Invalid	-100000	Đơn giá âm.
EC11	quantity	Valid	1 hoặc 2	Số lượng nguyên dương.
EC12	quantity	Invalid	Không truyền quantity	Thiếu số lượng.
EC13	quantity	Invalid	"abc"	Sai kiểu dữ liệu.
EC14	quantity	Invalid	0	Ngay dưới miền hợp lệ.
EC15	quantity	Invalid	-1	Số lượng âm.

Mã lớp	Biến tương ứng	Loại lớp	Giá trị đại diện	Ý nghĩa / Ghi chú kiểm thử
EC16	delete_confirmation	Valid	Confirm	Xác nhận xóa.
EC17	delete_confirmation	Valid	Cancel	Hủy xóa.
EC18	cart_state	Valid	Giò có 1 sản phẩm	Có thể render bảng giỏ.
EC19	cart_state	Valid	Giò không có sản phẩm	Kiểm tra empty state.
EC20	product_already_in_cart	Valid	false	Thêm dòng mới.
EC21	product_already_in_cart	Valid	true	Cộng dồn số lượng.
EC22	current_quantity	Valid	2	Giảm xuống 1 vẫn hợp lệ.
EC23	current_quantity	Invalid	1 rồi bấm giảm	Không được giảm xuống 0.
EC24	http_status_code	Valid	HTTP thành công	Thao tác hợp lệ được xử lý.
EC25	http_status_code	Invalid	HTTP lỗi xác thực/validation	Thao tác không hợp lệ bị từ chối.
EC26	api_response_payload	Valid	JSON giỏ hàng cập nhật	Dữ liệu đủ để render UI.
EC27	api_response_payload	Invalid	JSON thông báo lỗi	Dữ liệu lỗi phù hợp nghiệp vụ.
EC28	cart_table	Valid	Bảng có đủ 5 cột	Đúng yêu cầu hiển thị giỏ hàng.
EC29	empty_cart_state	Valid	Hình minh họa + thông báo	Đúng yêu cầu khi giỏ trống.
EC30	quantity_display / line_total	Valid	$2 \times 100000 = 200000$	Kiểm tra phép tính và hiển thị.
EC31	delete_confirmation_dialog	Valid	Dialog xác nhận xóa	Đúng yêu cầu bảo vệ thao tác xóa.
EC32	navigation_action	Valid	Quay về trang chủ	Đúng luồng tiếp tục mua sắm.
EC33	total_label	Valid	"Tổng cộng"	Nhãn tổng tiền đúng đặc tả.

2. Thiết kế tập Test Cases phân hoạch tương đương (Equivalence Partitioning Test Cases)

Tập test cases dưới đây bao phủ các lớp tương đương đã phân hoạch cho FR-07. Các biến trạng thái như giỏ trống/không trống, sản phẩm đã có trong giỏ và số lượng hiện tại được ghi trong Preconditions thay vì cột Input.

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit_price	quantity	del
TC01	Thêm sản phẩm mới vào giỏ hợp lệ	EC01, EC03, EC05, EC07, EC09, EC11, EC19, EC20, EC24, EC26, EC28, EC30, EC33	Giò hàng đang trống; sản phẩm chưa có trong giỏ.	Token user hợp lệ	add_item	1	Sản phẩm A	100000	1	N/A

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit_price	quantity	del
TC02	Thêm cùng một sản phẩm chỉ tăng số lượng	EC01, EC03, EC05, EC07, EC09, EC11, EC18, EC21, EC24, EC26, EC28, EC30	Giò đã có sản phẩm ID 1 với số lượng 1.	Token user hợp lệ	add_item	1	Sản phẩm A	100000	1	N/A
TC03	Từ chối truy cập giỏ hàng khi thiếu token	EC02, EC03, EC25, EC27	Người dùng chưa đăng nhập hoặc không gửi token.	Không truyền token	view_cart	N/A	N/A	N/A	N/A	N/A
TC04	Từ chối thêm sản phẩm thiếu số lượng	EC01, EC03, EC05, EC07, EC09, EC12, EC25, EC27	Sản phẩm hợp lệ, người dùng đã đăng nhập.	Token user hợp lệ	add_item	1	Sản phẩm A	100000	Không truyền	N/A
TC05	Từ chối số lượng sai kiểu	EC01, EC03, EC05, EC07, EC09, EC13, EC25, EC27	Sản phẩm hợp lệ, người dùng đã đăng nhập.	Token user hợp lệ	add_item	1	Sản phẩm A	100000	"abc"	N/A
TC06	Từ chối số lượng bằng 0	EC01, EC03, EC05, EC07, EC09, EC14, EC25, EC27	Sản phẩm hợp lệ, người dùng đã đăng nhập.	Token user hợp lệ	add_item	1	Sản phẩm A	100000	0	N/A
TC07	Từ chối	EC01, EC03,	Sản phẩm hợp lệ, người dùng	Token user hợp lệ	add_item	1	Sản phẩm A	100000	-1	N/A

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit_price	quantity	delivery
TC08	số lượng âm	EC05, EC07, EC09, EC15, EC25, EC27	đã đăng nhập.							
TC08	Từ chối sản phẩm thiếu ID	EC01, EC03, EC06, EC07, EC09, EC11, EC25, EC27	Người dùng đã đăng nhập, dữ liệu sản phẩm còn lại hợp lệ.	Token user hợp lệ	add_item	Không truyền	Sản phẩm A	100000	1	N/A
TC09	Từ chối đơn giá không hợp lệ	EC01, EC03, EC05, EC07, EC10, EC11, EC25, EC27	Người dùng đã đăng nhập, sản phẩm có ID và tên hợp lệ.	Token user hợp lệ	add_item	1	Sản phẩm A	-100000	1	N/A
TC10	Từ chối tên sản phẩm rỗng	EC01, EC03, EC05, EC08, EC09, EC11, EC25, EC27	Người dùng đã đăng nhập, ID và giá hợp lệ.	Token user hợp lệ	add_item	1	""	100000	1	N/A
TC11	Tăng số lượng bằng nút +	EC01, EC03, EC18, EC22, EC24, EC26, EC28, EC30	Giỏ có sản phẩm ID 1, số lượng hiện tại 1.	Token user hợp lệ	increase_quantity	1	Sản phẩm A	100000	2	N/A

Mã TC	Tên Test Case	Lớp tương đương phủ	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit_price	quantity	delivery_status
TC12	Giảm số lượng từ 2 xuống 1 bằng nút −	EC01, EC03, EC18, EC22, EC24, EC26, EC28, EC30	Giỏ có sản phẩm ID 1, số lượng hiện tại 2.	Token user hợp lệ	decrease_quantity	1	Sản phẩm A	100000	1	N/A
TC13	Không cho giảm số lượng dưới 1	EC01, EC03, EC18, EC23, EC25, EC27	Giỏ có sản phẩm ID 1, số lượng hiện tại 1; người dùng bấm −.	Token user hợp lệ	decrease_quantity	1	Sản phẩm A	100000	0	N/A
TC14	Xóa sản phẩm sau khi xác nhận	EC01, EC03, EC05, EC16, EC18, EC24, EC26, EC31	Giỏ có sản phẩm ID 1.	Token user hợp lệ	delete_item	1	Sản phẩm A	100000	1	Corrupted
TC15	Hủy thao tác xóa sản phẩm	EC01, EC03, EC05, EC17, EC18, EC31	Giỏ có sản phẩm ID 1.	Token user hợp lệ	delete_item	1	Sản phẩm A	100000	1	Cart Empty
TC16	Tiếp tục mua sắm quay về trang chủ	EC01, EC03, EC18, EC32	Người dùng đang ở màn hình giỏ hàng.	Token user hợp lệ	continue_shopping	N/A	N/A	N/A	N/A	N/A
TC17	Hiện thị trạng thái	EC01, EC03, EC19, EC29	Giỏ hàng của người dùng không có sản phẩm.	Token user hợp lệ	view_cart	N/A	N/A	N/A	N/A	N/A

Mã TC	Tên Test Case	Lớp tương đương phù	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit_price	quantity	del
	giỏ hàng trống									
TC18	Kiểm tra nhân tổng tiền chính xác	EC01, EC03, EC18, EC28, EC30, EC33	Giỏ hàng có ít nhất một sản phẩm.	Token user hợp lệ	view_cart	N/A	N/A	N/A	N/A	N/A
TC19	Kiểm tra robust API với thao tác giỏ hàng ngoài spec	EC01, EC04, EC25, EC27	Người dùng đã đăng nhập; giỏ hàng có thể rỗng hoặc không rỗng.	Token user hợp lệ	unsupported_action	N/A	N/A	N/A	N/A	N/A

Bước 4: Phân tích giá trị biên (Boundary Value Analysis - BVA)

Giải thích chi tiết từng bước (Step-by-Step Explanation)

Để xác định các giá trị biên nhạy cảm của tính năng **FR-07: Giỏ hàng (Shopping Cart)**, em đã thực hiện phân tích theo các bước sau:

1. **Xác định các biến có tính thứ tự hoặc khoảng số:** Các biến phù hợp BVA gồm **quantity**, **current_quantity** khi bấm nút giảm, và **cart_items_count** để kiểm tra chuyển đổi giữa empty state và bảng giỏ hàng.
2. **Xác định các điểm biên (Boundaries) cho từng biến:** Ràng buộc rõ nhất của FR-07 là số lượng tối thiểu **1**; ngoài ra số dòng giỏ hàng có biên quan trọng tại **0** sản phẩm.
3. **Lựa chọn các điểm kiểm thử biên nhạy cảm:** Chọn **quantity = 0, 1, 2**, **current_quantity = 1, 2** và **cart_items_count = 0, 1, 2** để kiểm tra ranh giới giữa không hợp lệ/hợp lệ và giữa empty state/bảng giỏ hàng.

1. Phân tích giá trị biên của các biến số/khoảng số

- Biến **quantity** (Khoảng hợp lệ: **[1, +∞)**):**
 - LB = 1:** Số lượng tối thiểu hợp lệ.
 - LB + 1 = 2:** Số lượng hợp lệ ngay phía trong miền hợp lệ.
 - LB - 1 = 0:** Số lượng không hợp lệ ngay dưới biên.
- Biến **current_quantity** khi bấm giảm (Khoảng hợp lệ sau thao tác: **[1, +∞)**):**
 - LB = 1:** Nếu đang ở **1**, hệ thống không được giảm tiếp thành **0**.
 - LB + 1 = 2:** Nếu đang ở **2**, bấm giảm còn **1** là hợp lệ.
- Biến **cart_items_count** (Số dòng sản phẩm trong giỏ, khoảng hợp lệ quan sát được: **[0, +∞)**):**
 - LB = 0:** Giỏ trống, phải hiển thị empty state.
 - LB + 1 = 1:** Có một dòng sản phẩm, phải hiển thị bảng giỏ hàng.

- $LB + 2 = 2$: Có nhiều dòng sản phẩm, tổng cộng phải cộng đúng nhiều dòng.

2. Thiết kế tập Test Cases giá trị biên (Boundary Value Test Cases)

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên đề (Preconditions)	authorization_token	cart_action	product_id	product_name	unit
TC-BVA-01	Thêm sản phẩm với $quantity = 0$	$quantity = LB - 1 = 0$	Người dùng đã đăng nhập; sản phẩm hợp lệ.	Token user hợp lệ	add_item	1	Sản phẩm A	1000
TC-BVA-02	Thêm sản phẩm với $quantity = 1$	$quantity = LB = 1$	Người dùng đã đăng nhập; sản phẩm chưa có trong giỏ.	Token user hợp lệ	add_item	1	Sản phẩm A	1000
TC-BVA-03	Thêm sản phẩm với $quantity = 2$	$quantity = LB + 1 = 2$	Người dùng đã đăng nhập; sản phẩm chưa có trong giỏ.	Token user hợp lệ	add_item	1	Sản phẩm A	1000
TC-BVA-04	Bấm giảm khi $current_quantity = 1$	$current_quantity = LB = 1$	Giỏ có sản phẩm ID 1, số lượng hiện tại 1.	Token user hợp lệ	decrease_quantity	1	Sản phẩm A	1000
TC-BVA-05	Bấm giảm khi $current_quantity = 2$	$current_quantity = LB + 1 = 2$	Giỏ có sản phẩm ID 1, số lượng hiện tại 2.	Token user hợp lệ	decrease_quantity	1	Sản phẩm A	1000
TC-BVA-06	Xem giỏ với $cart_items_count = 0$	$cart_items_count = LB = 0$	Giỏ hàng đang trống.	Token user hợp lệ	view_cart	N/A	N/A	N/A

Mã TC	Tên Test Case	Biên kiểm thử	Điều kiện tiên để (Preconditions)	authorization_token	cart_action	product_id	product_name	un
TC-BVA-07	Xem giỏ với cart_items_count = 1	cart_items_count = LB+1 = 1	Giỏ hàng có đúng 1 dòng sản phẩm.	Token user hợp lệ	view_cart	N/A	N/A	N/A
TC-BVA-08	Xem giỏ với cart_items_count = 2	cart_items_count = LB+2 = 2	Giỏ hàng có 2 dòng sản phẩm khác nhau.	Token user hợp lệ	view_cart	N/A	N/A	N/A

Bước 5: Phân tích khoảng trống AI (AI Gap Analysis)

1. Các kịch bản/lỗi kiểm thử mà AI đã bỏ sót

- **Bypass validation số lượng qua API trực tiếp:** UI có thể dùng nút +/- và input số để hạn chế giá trị, nhưng API `POST /api/cart` vẫn cần tự kiểm tra `quantity` rỗng, sai kiểu, bằng 0 hoặc âm.
- **Cộng dồn sản phẩm trùng thay vì tạo dòng mới:** AI dễ kiểm tra thêm sản phẩm mới nhưng bỏ qua yêu cầu đặc thù rằng thêm cùng sản phẩm phải tăng số lượng trên dòng hiện có.
- **Hành vi xóa cần xác nhận:** AI thường kiểm tra sản phẩm có bị xóa hay không, nhưng bỏ sót điều kiện UI phải hiển thị dialog xác nhận và phải giữ nguyên giỏ khi người dùng hủy.

2. Các sự nhầm lẫn, ảo giác và thiếu sót của AI trong quá trình thiết kế (AI Critique)

- **Nhầm State Input thành Direct Input:** Các trạng thái như `cart_state`, `product_already_in_cart` và `current_quantity` không phải trường body độc lập mà là điều kiện tiên đề cần thiết lập qua thao tác black-box.
- **Implementation Bias về thông điệp lỗi:** Nếu AI ghi cứng chuỗi JSON cụ thể, test case sẽ phụ thuộc implementation thay vì yêu cầu nghiệp vụ. Expected Output nên mô tả hành vi như "bị từ chối do số lượng không hợp lệ".
- **Bỏ sót yêu cầu hiển thị nhỏ nhưng bắt buộc:** Nhãn **"Tổng cộng"** và empty-cart illustration/message là yêu cầu chấm điểm rõ ràng, nhưng AI có thể chỉ tập trung API mà không kiểm tra UI.

3. Giải thích nguyên nhân AI gặp các hạn chế trên

- **Đặc tả FR-07 kết hợp cả UI và API:** Một số yêu cầu nằm ở giao diện, một số nằm ở API, nên AI dễ thiên lệch sang một phía nếu prompt không nhắc rõ black-box.
- **Luồng giỏ hàng phụ thuộc trạng thái trước đó:** Các ca như sản phẩm đã có trong giỏ hoặc số lượng hiện tại bằng 1 cần precondition, không thể chỉ nhìn một request đơn lẻ.
- **Thiếu quan sát thực thi trong giai đoạn thiết kế:** Ở bước thiết kế, Actual Output và Pass/Fail chưa được điền; các lỗi thật chỉ được tổng hợp sau khi chạy test qua UI/API công khai.